Communication and Storage-Friendly Bidirectional Multi-hop CPA Secure Proxy Re-encryption from Supersingular Isogenies

Manas Jana[1], Ratna Dutta[1], and Sourav Mukhopadhyay[1]

Department of Mathematics, Indian Institute of Technology, Kharagpur, West Bengal, India
`mjana@kgpian.iitkgp.ac.in`, `ratna@maths.iitkgp.ac.in`, `sourav@maths.iitkgp.ac.in`

Abstract. *Proxy re-encryption* (PRE) is an essential cryptographic primitive for managing secure access delegation in outsourced data environments, particularly public cloud systems. PRE is a *public key encryption* (PKE) with two additional algorithms - (i) re-encryption key generation by which a proxy server generates a re-encryption key; (ii) re-encryption algorithm by which the proxy server can transform the ciphertext under the delegator's public key to a ciphertext under the delegatee's public key enabling the delegatee to decrypt the message originally intended for the delegator. With the advent of quantum computing, a pressing need arises to design PRE schemes based on quantum-resistant assumptions. This paper addresses this requirement by presenting the *first* construction of a *bidirectional* PRE (bPRE) from supersingular isogenies. Our bPRE is built upon the commutative supersingular isogeny-based PKE scheme MSimS, a variant of the isogeny-based PKE scheme SimS [12] and achieves security against *chosen-plaintext attack* (CPA) in the standard model under the hardness of the *commutative supersingular isogeny decisional Diffie-Hellman* (CSSIDDH) problem. The resultant bPRE supports efficient re-encryption of ciphertexts by the proxy server for the delegator as well as the delegatee and inherits the multi-hop property, enabling chainable delegation of access rights. Significantly, our isogeny-based bPRE is asymptotically efficient, offering an efficient reduction in bandwidth consumption compared to current lattice-based proposals in terms of key size and ciphertext size. This makes the scheme a highly compact and practical candidate for post-quantum cloud security. Furthermore, our PKE scheme MSimS is of independent interest which is proven to be CPA secure under the hardness of the CSSIDDH problem and secure against *chosen ciphertext attack* (CCA) under the hardness of the CSSIDDH problem and the *commutative supersingular isogeny knowledge of exponent* (CSSIKOE) problem.

Keywords: Post-quantum cryptography · Supersingular isogenies · Proxy Re-Encryption · Access delegation

1 Introduction

The rise of cloud computing has changed the way the traditional technology works, promising substantial enhancements in performance, faster business processes and most importantly significant reductions in costs. New types of risks to privacy and security arise with these expectations. When moving from a data owner's fully controlled infrastructure to third-party-administered public cloud resources, the threat paradigm changes in an extensive manner. The most common security model in modern cloud systems is one that focuses on the perimeter and tries to keep unauthorised users from getting to internal servers and databases that store user data. To address these threats, countermeasures such as internal policies and more robust access controls are frequently recommended which only serve to highlight a basic issue with trust. Traditional public key encryption cannot resolve the issue of secure access delegation in the cloud without adopting sophisticated key management mechanisms.

Proxy re-encryption (PRE). *Proxy re-encryption* (PRE) introduced by Blaze et al. [2], is a prominent candidate that not only provides secure access control systems where the protected data is stored externally, but also enables dynamic delegation to encrypted data. Using PRE, a user can store encrypted data in the cloud and share the data with the authorised users keeping the data confidential to the unauthorised users and the cloud provider itself. In addition, PRE allows a user to delegate her decryption rights to other authorised users with the help of a semi-trusted proxy server. A proxy server can convert ciphertexts encrypted with the delegator's public key into new ciphertexts for the delegatee without the proxy being able to extract the underlying plaintexts and any knowledge of the secret keys of the delegators and delegatees. This transformation property of PRE makes it usefull in many applications including secure access control [34], distributed file systems [1], simplification of key distribution [2], secure email forwarding [19], digital right management (DRM) of Apple's iTunes [32], multicast [7] and privacy for public transportation [15]. We describe below how a PRE can provide an elegant solution for secure multi-device cloud storage access [29].

Consider a user Alice has multiple devices (for example, a laptop, a mobile, a tablet, etc.), each equipped with a public-secret key pair (generated by algorithm KeyGen using the public parameter generated from Setup algorithm) and she wants to be able to access her encrypted files (generated by algorithm Enc) from any of these devices. Suppose she encrypts a file using the public key of her primary device and uploads $\mathsf{ct}_{\mathsf{primary}}$ to the cloud. The cloud server acts as a proxy server in PRE that enables the primary device (delegator) and secondary device (delegatee) to generate a special re-encryption key (generated using the algorithm $\mathsf{ReKeyGen}$) by employing a three-party protocol among the proxy server, the delegator and the delegatee. Using this re-encryption key, the cloud server can re-encrypt the ciphertext $\mathsf{ct}_{\mathsf{primary}}$ (generated under the public key of the primary device) to a ciphertext $\mathsf{ct}_{\mathsf{secondary}}$ (generated under a public key of the secondary device), allowing Alice to decrypt $\mathsf{ct}_{\mathsf{secondary}}$ with the secret key of her secondary device (using the algorithm Dec). The cloud is oblivious to the secret keys of Alice's devices and always remains unaware of the original file that was encrypted with the public key of the primary device even with the knowledge of the re-encryption key.

Unidirectional vs. Bidirectional PRE. On the basis of the direction of the transformation, PRE can be categorised into two types - *unidirectional* and *bidirectional* [17]. In a unidirectional PRE [1, 4, 8, 14, 20, 28], the semi-trusted proxy server can convert the ciphertexts under the public key of a delegator to the ciphertexts under the public key of a delegatee only, not the reverse. In contrast, the semi-trusted proxy server in a bidirectional PRE [2,11,23,29,37,38] is capable of converting ciphertexts under delegator's the public key to the ciphertext under delegatee's public key and vice versa.

Multi-hop vs Single-hop PRE. PRE schemes can be classified into *multi-hop* and *single-hop* according to the number of transformations of the ciphertexts. In a multi-hop PRE [2, 4, 23, 37, 38], the semi-trusted proxy can transform the ciphertexts multiple times whereas in a single-hop PRE [1,14,28], the proxy can transform the ciphertext only once [17].

Related works. The PRE scheme was introduced by Blaze et al. [2] in 1998 from modified ElGamal encryption under the hardness of the *Decesional Diffie-Hellman* (DDH) assumption. Later in 2003, Ivan and Dodis [17] proposed a generic construction of unidirectional and single-hop PRE and proposed a security model for *chosen ciphertext attacks* (CCA) that allows the delegatee to use the proxy as an oracle. Their security framework does not provide enough security for various applications [28]. In 2006, Ateniese et al. [1] formalised the PRE syntax with a new definition of *chosen plaintext attack* (CPA) security model and provided a construction of a CPA secure unidirectional single-hop PRE based on bilinear pairing which is secure under the hardness of *extended Decesional Bilinear Diffie-Hellman* (eDBDH) assumption. In 2007, Canetti and Hohenberger [4] formulated the definition of CPA security and CCA security for bidirectional PRE, introduced a CCA secure bidirectional pairing-based multi-hop PRE by integrating a one-time signature to authenticate a segment of the ciphertext that remains intact during the re-encryption step and achieved security under the hardness of DBDH assumption. They have also shown the relationships between the game-based and simulation-based security definitions of the bidirectional PRE scheme. In 2008, Deng et al. [10] designed a CCA secure pairing-free bidirectional multi-hop PRE from modified hashed ElGamal encryption and a modified Schnorr signature and achieved security in the random oracle model under the hardness of *divisible computation Diffie-Hellman* (DCDH) problem, a variant of the *computational Diffie-Hellman* (CDH) problem. In 2009, Shao and Cao [28] constructed a CCA secure pairing-free unidirectional single-hop PRE from the DDH assumption over $\mathbb{Z}^*_{N^2}$ ($N = pq$ being a safe prime such that $p = 2p' + 1, q = 2q' + 1$ and p, p', q, q' are primes) coupling the Fujisaki-Okamoto [13] transformation with the CPA secure public key encryption of Bresson et al. [3] with double trapdoors. The scheme was broken by Chow et al. [8] in 2010 and an improved variant of pairing-free CCA secure unidirectional single-hop PRE was proposed in the random oracle model under the hardness of the CDH assumption.

Post-quantum PRE. In the coming years, the expected growth of quantum computers will become a huge security threat, leading to the construction of quantum-resistant cryptographic protocols. In 2010, Xagawa and Tanaka [38] introduced a CPA secure bidirectional multi-hop PRE based on the hardness of the *learning with errors* (LWE) problem from a lattice-based public key encryption of Peikert et al. [25] which is a variant of Regev's public key encryption [27]. In 2014, Kirshanova [20] proposed a CCA secure unidirectional single-hop PRE under the LWE assumption applying the lattice-based trapdoor delegation technique of Micciancio and Peikert [21]. Later Fan et al. [11] found a security flaw in the security proof of [20] and gave a unidirectional multi-hop PRE in a tag-based CCA security model under the LWE assumption, associating every ciphertext with a randomly chosen tag in the encryption algorithm. In 2015, Nunez et al. [23] first came up with NTRUReEncrypt, a CPA secure bidirectional multi-hop PRE based on lattice-based public key encryption NTRU [16].

1.1 Our Contributions

Although lattice-based PRE constructions are promising, they face inherent challenges due to error accumulation with each additional operation. Unlike the lattice-based post-quantum approach, the isogeny-based cryptography provides compact key size and ciphertext length. To the best of our knowledge, there is no construction of PRE based on isogeny. We propose the *first* bidirectional multi-hop PRE scheme bPRE from supersingular isogenies. The basic building block of our bPRE is MSimS, a variant of the supersingular isogeny-based IND-CCA secure *public key encryption* (PKE) scheme SimS of Fouotsa et al. [12], which is secure under the hardness of the *commutative supersingular isogeny decisional Diffie-Hellman* (CSSIDDH) assumption and the *commutative supersingular isogeny knowledge of exponent* (CSSIKOE) assumption. More concretely, our contributions in this paper are two-fold.

– We start with the PKE scheme SimS [12], identifying the difficulties in plugging SimS directly to design our bPRE and propose our PKE scheme MSimS = (Setup, KeyGen, Enc, Dec) with appropriate modifications in the encryption and decryption algorithms Enc and Dec of SimS respectively, making it suitable for bPRE construction. Similar to SimS, our modified scheme MSimS employs an elliptic curve point-finding algorithm of specific order and a randomized encoding function during the encryption as well as decryption process together with one invocation of the Pohlig-Hellman algorithm to solve the discrete logarithm problem during decryption. Our proposed scheme MSimS not only retains the same complexity as that of SimS in terms of computation, communication and storage overhead, but also exhibits exactly the same security as that of SimS. We provide rigorous security analysis of MSimS in standard security framework and show that MSimS is IND-CPA secure under the hardness of the CSSIDDH assumption

and IND-CCA secure under the hardness of the CSSIDDH and CSSIKOE assumptions.
- Finally, we design our $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$. The algorithms Setup, KeyGen, Enc and Dec of our bPRE are exactly identical to those of our MSimS. We design the algorithm ReKeyGen for generating re-encryption key by employing a three-party protocol among the semi-trusted proxy server, the delegator and the delegatee. In addition the algorithm ReEnc is designed for re-encrypting the delegator's ciphertext. We prove the CPA security of our scheme bPRE under the CSSIDDH assumption.

We compare the lattice-based CPA secure multi-hop bidirectional PRE scheme NTRUReEncrypt of Nunez et al. [23] with our bidirectional PRE scheme bPRE in Table 1 which exhibits that our bPRE outperforms in terms of secret key size, public key size, ciphertext size and re-encryption key size. To be more specific, the secret key as well as the re-encryption key of our proposed bPRE is a vector from $\{-s, \ldots, s\}^n$ and has size $O(n)$ each and the public key, being an element of a finite field $\mathbb{F}_p$, has size $O(\log_2 p)$. Here $p = 2^r \ell_1 \cdots \ell_n - 1$ is a prime number, $\ell_1, \ldots, \ell_n$ are small odd primes, $r \geq 3$ is an integer satisfying $\lambda + 2 \leq r \leq \frac{1}{2} \log_2 p$, s is an integer satisfying $(2s + 1)^n \geq |\mathsf{cl}(\mathcal{O})|$, $\mathsf{cl}(\mathcal{O})$ is the ideal class group of the endomorphism ring $\mathcal{O} = \mathsf{End}_{\mathbb{F}_p}(E_0)$ over $\mathbb{F}_p$ and $E_0 : y^2 = x^3 + x$ is the supersingular elliptic curve over $\mathbb{F}_p$. In contrast, the sizes of the secret key, the public key and the re-encryption key of NTRUReEncrypt are $O(n \log_2 p)$ each. The ciphertext of our bPRE consists of two elements of $\mathbb{F}_p$ and therefore has size $O(\log_2 p)$ whereas the size of the ciphertext in NTRUReEncrypt is $O(n \log_2 p)$.

Table 1. Comparison of communication and storage overhead between existing quantum-safe CPA secure bPRE schemes

Scheme	$\lvert\mathsf{sk}\rvert$	$\lvert\mathsf{pk}\rvert$	$\lvert\mathsf{ct}\rvert$	$\lvert\mathsf{rk}\rvert$	Assumption
NTRUReEncrypt [23]	$O(n \log_2 p)$	$O(n \log_2 p)$	$O(n \log_2 p)$	$O(n \log_2 p)$	LWE
Our bPRE	$O(n)$	$O(\log_2 p)$	$O(\log_2 p)$	$O(n)$	CSSIDDH

$|\mathsf{sk}|$ = size of the secret key, $|\mathsf{pk}|$ = size of the public key, $|\mathsf{ct}|$ = size of the ciphertext, $|\mathsf{rk}|$ = size of a re-encryption key, $p = 2^r \ell_1 \cdots \ell_n - 1$ is a prime number where ℓ_i's are small odd primes, $r \geq 3$ is an integer satisfying $\lambda + 2 \leq r \leq \frac{1}{2} \log_2 p$ and λ is the security parameter.

Our bPRE requires five group action (GA) computation, 1 GA for KeyGen, 2 GA for Enc, 1 GA for Dec and 1 GA for ReEnc. The computation complexity for class group computation is polynomial in $\log_2 p$. Also, our bPRE also requires finding two points of order 2^r on elliptic curves over $\mathbb{F}_p$ by a deterministic algorithm with computational complexity $O(\log_2 p)$, 1 during Enc and 1 during Dec. Apart from that, one discrete logarithm is calculated during Dec by the Pohlig-Hellman algorithm. The computational complexity of the Pohlig-Hellman is $O(r^2)$. These computational costs have little effect on the overall encryption, decryption and re-encryption process in our bPRE. Similar to NTRUReEncrypt, we also need a three-party protocol during re-encryption key generation with communication complexity $O(n)$.

1.2 Technical Overview

CPA secure bidirectional proxy re-encryption bPRE. To construct an isogeny-based bPRE, we start with the isogeny-based IND-CCA secure public key encryption scheme SimS [12] where a user samples its secret key $\mathsf{sk} = (a_1, \ldots, a_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$ and sets its public key $\mathsf{pk} = E_A$ by computing $E_A = [\mathfrak{a}]E_0 \in \mathscr{E}_p(\mathscr{O})$. Here $\mathscr{E}_p(\mathscr{O})$ is the set of supersingular elliptic curves E defined over $\mathbb{F}_p$ with the endomorphism ring $\mathsf{End}_{\mathbb{F}_p}(E) = \mathscr{O}$, $[\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}]$ is the ideal class , $\mathfrak{l}_i = \langle \ell_i, \pi - 1\rangle$ is the ideals for $i = 1, \ldots, n$ and π is the Frobenius map, E_0 is the base elliptic curve $y^2 = x^3 + x$ over $\mathbb{F}_p$, $p = 2^r\ell_1 \cdots \ell_n - 1$ is a prime, $\ell_1, \ldots, \ell_n$ are small odd primes. The ciphertext in SimS corresponding to a message $m \in \mathbb{Z}_{2^{r-2}}$ under a public key $\mathsf{pk} = [\mathfrak{a}]E_0$ is $\mathsf{ct}_{\mathsf{SimS}} = (E_3 = [\mathfrak{b}]E_0, f_{[\mathfrak{b}][\mathfrak{a}]E_0}(x([2m+1]P_{[\mathfrak{b}][\mathfrak{a}]E_0})))$ where the ideal classes $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$, $f_{[\mathfrak{b}][\mathfrak{a}]E_0}$ is a randomizing function corresponding to the elliptic curve $E_4 = [\mathfrak{b}][\mathfrak{a}]E_0$ and $x(P_{[\mathfrak{b}][\mathfrak{a}]E_0})$ is the x-coordinate of a point $P_{[\mathfrak{b}][\mathfrak{a}]E_0} \in E_4$ of order 2^r. The function $f_E : \mathbb{F}_p \to \mathbb{F}_p$ indexed by supersingular elliptic curves E is a bijection with its inverse $g_E : \mathbb{F}_p \to \mathbb{F}_p$ which satisfies the following:

- f_E and g_E are efficiently computable when E is given.
- $f_E(x)$ is indistinguishable from a random element of $\mathbb{F}_p$ to a probabilistic polynomial time (PPT) adversary $\mathscr{A}$ having no access of x and E.
- A PPT adversary having no access of $x \in \mathbb{F}_p$ and the elliptic curve E, cannot compute $f_E(R(x))$ from $f_E(x)$ where $R(x)$ is a rational function in $\mathbb{F}_p(X)$.

Since both the components of $\mathsf{ct}_{\mathsf{SimS}}$ involve the random ideal class $[\mathfrak{b}]$, a semi-trusted proxy server cannot re-encrypt the ciphertext without the knowledge of the random ideal class $[\mathfrak{b}]$. We modify the ciphertext corresponding to a message $m \in \mathbb{Z}_{2^{r-2}}$ under a public key $\mathsf{pk} = [\mathfrak{a}]E_0$ as

$$\mathsf{ct} = (E_4 = [\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0}))$$

where the ideal classes $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$, $f_{[\mathfrak{b}]E_0}$ is the randomizing function corresponding to the elliptic curve $E_3 = [\mathfrak{b}]E_0$ and $x(P_{[\mathfrak{b}]E_0})$ is the x-coordinate of the point $P_{[\mathfrak{b}]E_0} \in E_3$ of order 2^r. A semi-trusted proxy server can re-encrypt a ciphertext $\mathsf{ct}_i = ([\mathfrak{b}][\mathfrak{a}^{(i)}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$ under a delegator's public key $\mathsf{pk}_i = [\mathfrak{a}^{(i)}]E_0$ into the ciphertext $\mathsf{ct}_j = ([\mathfrak{b}][\mathfrak{a}^{(j)}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$ under a delegatee's public key $\mathsf{pk}_j = [\mathfrak{a}^{(j)}]E_0$ using the re-encryption key $\mathsf{rk}_{i\leftrightarrow j} = [\mathfrak{a}^{(\mathrm{j})}(\mathfrak{a}^{(i)})^{-1}]$. Here the re-encryption key $\mathsf{rk}_{i\leftrightarrow j} = [\mathfrak{a}^{(\mathrm{j})}(\mathfrak{a}^{(i)})^{-1}]$ is generated by a three-party protocol among the semi-trusted proxy server, the delegator and the delegatee. The proxy server cannot reconstruct the message $m \in \mathbb{Z}_{2^{r-2}}$ even with the knowledge of the re-encryption key $\mathsf{rk}_{i\leftrightarrow j} = [\mathfrak{a}^{(\mathrm{j})}(\mathfrak{a}^{(i)})^{-1}]$.

CPA Security of bPRE. The CPA security of our bPRE relies on the CSSIDDH assumption. Given a CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ for some $[\mathfrak{c}], [\mathfrak{d}] \in \mathsf{cl}(\mathscr{O})$, a CSSIDDH distinguisher $\mathscr{B}$ can be constructed using a CPA adversary $\mathscr{A}$ against our bPRE who can decide $E = [\mathfrak{c}][\mathfrak{d}]E_0$ or $E = [\mathfrak{r}]E_0$ for some random $[\mathfrak{r}] \in \mathsf{cl}(\mathscr{O})$. In the challenge phase the CSSIDDH distinguisher $\mathscr{B}$ sets the challenge public key $\mathsf{pk}_{i^*} = [\mathfrak{c}]E_0$ using the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ and chooses a bit $\mu \xleftarrow{\$} \{0, 1\}$. The distinguisher $\mathscr{B}$ sets the challenge ciphertext $\mathsf{ct}^* = (E, f_{[\mathfrak{d}]E_0}(x([2m_\mu + 1]P_{[\mathfrak{d}]E_0})))$ from the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ where (m_0, m_1) is the challenge message pair and $P_{[\mathfrak{d}]E_0} \in [\mathfrak{d}]E_0$ is a point of order 2^r.

- If $E = [\mathfrak{c}][\mathfrak{d}]E_0$ in the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$, then the simulation of the challenge ciphertext $\mathsf{ct}^* = (E, f_{[\mathfrak{d}]E_0}(x([2m_\mu + 1]P_{[\mathfrak{d}]E_0}))) = ([\mathfrak{c}][\mathfrak{d}]E_0, f_{[\mathfrak{d}]E_0}(x([2m_\mu + 1]P_{[\mathfrak{d}]E_0})))$ is exactly identical with the original protocol and the CPA adversary $\mathscr{A}$ can guess the bit μ with high probability.
- If $E = [\mathfrak{r}]E_0$ in the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ for some random $[\mathfrak{r}] \in \mathsf{cl}(\mathscr{O})$, then the challenge ciphertext $\mathsf{ct}^* = ([\mathfrak{r}]E_0, f_{[\mathfrak{d}]E_0}([2m_\mu + 1]))$ and CPA adversary $\mathscr{A}$ can guess the bit μ with probability $\frac{1}{2}$ in this case.

Thus, if there exists a CPA adversary $\mathscr{A}$ against our bPRE then we can construct a CSSIDDH distinguisher $\mathscr{B}$. Consequently, we have the following theorem.

Theorem 1 (Informally). *Under the CSSIDDH assumption our* bPRE *is CPA secure.*

IND-CCA secure public key encryption MSimS**.** The underlying encryption scheme MSimS of our bPRE is a variant of SimS [12]. In MSimS, a sender encrypts a message $m \in \mathbb{Z}_{2^{r-2}}$ under a receiver's public key $\mathsf{pk} = [\mathfrak{a}]E_0$ as $\mathsf{ct} = (E_4 = [\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$ where $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$, $P_{[\mathfrak{b}]E_0} \in [\mathfrak{b}]E_0$ is a point of order 2^r, E_0 is the base supersingular elliptic curve $y^2 = x^3 + x$ over $\mathbb{F}_p$, $p = 2^r \ell_1 \cdots \ell_n - 1$ is a prime, $\ell_1, \ldots, \ell_n$ are small odd primes and r is an integer. The receiver extracts E_4 from the ciphertext ct and checks whether it is supersingular or not. If E_4 is supersingular, then the receiver computes the ideal class $[\mathfrak{a}^{-1}] = \prod_{i=1}^{n} [\mathfrak{l}_i^{-a_i}]$ with its secret key $\mathsf{sk} = (a_1, \ldots, a_n)$ where $\mathfrak{l}_i = \langle \ell_i, \pi - 1 \rangle$ for $i = 1, \ldots, n$ and π is the Frobenius map. Also it computes the class group action $E_3 = [\mathfrak{a}^{-1}]E_4$ and finds a point $P_{E_3} \in E_3$ of order 2^r. Observe that $E_3 = [\mathfrak{a}^{-1}]E_4 = [\mathfrak{a}^{-1}][\mathfrak{b}][\mathfrak{a}]E_0 = [\mathfrak{b}]E_0$ by commutativity of the ideal class group $\mathsf{cl}(\mathscr{O})$. The receiver can find the x-coordinate of the point $P_3 = [2m + 1]P_{[\mathfrak{b}]E_0}$ by calculating $x(P_3) = g_{E_3}(f_{E_3}(x([2m + 1]P_{[\mathfrak{b}]E_0}))$ where $g_{E_3} = f_{E_3}^{-1}$. The receiver gets $M \in \mathbb{Z}_{2^r}^{\times}$ by applying Pohlig-Hellman algorithm on $x(P_{E_3})$ and $x(P_3)$. If $2^{r-1} < M$ its sets $M' = 2^r - M$ otherwise $M' = M$. Finally, it gets the message $m = \frac{M'-1}{2}$.

IND-CCA secure public key encryption MSimS**.** The underlying encryption scheme MSimS of our bPRE is a variant of SimS [12]. In MSimS, a sender encrypts

a message $m \in \mathbb{Z}_{2^{r-2}}$ under a receiver's public key $\mathsf{pk} = [\mathfrak{a}]E_0$ as $\mathsf{ct} = (E_4 = [\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$ where $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$, $P_{[\mathfrak{b}]E_0} \in [\mathfrak{b}]E_0$ is a point of order 2^r, E_0 is the base supersingular elliptic curve $y^2 = x^3 + x$ over $\mathbb{F}_p$, $p = 2^r \ell_1 \cdots \ell_n - 1$ is a prime, $\ell_1, \ldots, \ell_n$ are small odd primes and r is an integer. The receiver extracts E_4 from the ciphertext ct and checks whether it is supersingular or not. If E_4 is supersingular, then the receiver computes the ideal class $[\mathfrak{a}^{-1}] = \prod_{i=1}^{n} [\mathfrak{l}_i^{-a_i}]$ with its secret key $\mathsf{sk} = (a_1, \ldots, a_n)$ where $\mathfrak{l}_i = \langle \ell_i, \pi - 1 \rangle$ is the ideal for $i = 1, \ldots, n$ and π is the Frobenius map. Also it computes the class group action $E_3 = [\mathfrak{a}^{-1}]E_4$ and finds a point $P_{E_3} \in E_3$ of order 2^r. Observe that $E_3 = [\mathfrak{a}^{-1}]E_4 = [\mathfrak{a}^{-1}][\mathfrak{b}][\mathfrak{a}]E_0 = [\mathfrak{b}]E_0$ by commutativity of the ideal class group $\mathsf{cl}(\mathscr{O})$. The receiver can find the x-coordinate of the point $P_3 = [2m+1]P_{[\mathfrak{b}]E_0}$ by calculating $x(P_3) = g_{E_3}(f_{E_3}(x([2m+1]P_{[\mathfrak{b}]E_0}))$ where $g_{E_3} = f_{E_3}^{-1}$. The receiver gets $M \in \mathbb{Z}_{2^r}^{\times}$ by applying Pohlig-Hellman algorithm on $x(P_{E_3})$ and $x(P_3)$. If $2^{r-1} < M$ its sets $M' = 2^r - M$ otherwise $M' = M$. Finally, it gets the message $m = \frac{M'-1}{2}$.

IND-CCA Security of MSimS. To prove the IND-CCA security of our public key encryption MSimS, we first established that MSimS is IND-CPA secure under the CSSIDDH assumption. Then we use a reduction method to prove the IND-CCA security of MSimS under an additional assumption Commutative Supersingular Isogeny Knowledge of Exponent (CSSIKOE). We follow the same technique of CPA security proof of our bPRE for proving the IND-CPA security of the public key encryption scheme MSimS. In the construction of MSimS the ciphertext corresponding to a message $m \in \mathbb{Z}_{2^{r-2}}$ under a public key $\mathsf{pk} = [a]E_0$ is $\mathsf{ct} = ([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$. The CSSIKOE assumption states that the only effective way to compute a valid ciphertext is to first fix the ideal class $[\mathfrak{b}]$ and then compute $\mathsf{ct} = ([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$ which implies a IND-CCA adversary can determine a challenge ciphertext ct^* is encrypted from a message m_0 or m_1 without using the decryption oracle. This will contradict the IND-CPA security of the public key encryption scheme MSimS. Hence, we have the following theorems.

Theorem 2 (Informally). *Under the CSSIDDH assumption, the public key encryption* MSimS *is IND-CPA secure.*

Theorem 3 (Informally). *Under the CSSIDDH and CSSIKOE assumptions, the public key encryption* MSimS *is IND-CCA secure.*

2 Preliminaries

Notations: We write $x \xleftarrow{\$} A$ for selecting an uniformly random element x from a set A. We denote by $\overline{K}$ the algebraic closure of a field K. Let $\mathbb{F}_p$ represents a finite field of p elements for a prime number p and $x(P)$ stands for the x-coordinate of a point $P \in E$ for an elliptic curve E. We call a function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}$ a negligible function if for every polynomial $\mathsf{poly}(x)$ in x, there exists a positive integer N such that $\mathsf{negl}(x) < \frac{1}{\mathsf{poly}(x)} \forall x > N$ where $\mathbb{N}$ is the set of natural numbers and $\mathbb{R}$ is the set of real numbers.

2.1 Public Key Encryption

Definition 1 (Public key encryption [18]). *A Public Key Encryption (PKE) scheme is a quadruple of algorithms* $\mathsf{PKE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *with a message space* $\mathscr{M}$ *that satisfies the following.*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}_{\mathsf{PKE}}$. *Taking a security parameter* 1^λ *as an input, a trusted authority runs this probabilistic polynomial time (PPT) algorithm, generates the public parameter* $\mathsf{pp}_{\mathsf{PKE}}$ *and makes* $\mathsf{pp}_{\mathsf{PKE}}$ *public to all.*
- $\mathsf{KeyGen}(\mathsf{pp}_{\mathsf{PKE}}) \to (\mathsf{sk}, \mathsf{pk})$. *Taking the public parameter* $\mathsf{pp}_{\mathsf{PKE}}$ *as input, a user runs this PPT algorithm and generates a public key* pk *and the corresponding secret key* sk. *The public key* pk *is made public and the secret key* sk *is kept secret to the user.*
- $\mathsf{Enc}(\mathsf{pp}_{\mathsf{PKE}}, m, \mathsf{pk}) \to \mathsf{ct}$. *On input a message* $m \in \mathscr{M}$, *the public key* pk *of a receiver and the public parameter* $\mathsf{pp}_{\mathsf{PKE}}$, *a sender runs this PPT algorithm and generates a ciphertext* ct *which is sent over a public channel to the receiver.*
- $\mathsf{Dec}(\mathsf{pp}_{\mathsf{PKE}}, \mathsf{ct}, \mathsf{sk}) \to m/\perp$. *Given a ciphertext* ct *and the public parameter* $\mathsf{pp}_{\mathsf{PKE}}$, *a receiver with its secret key* sk *runs this deterministic algorithm and returns either a plaintext* m *or* $\perp$, *indicating decryption failure.*

Correctness. A public key encryption scheme $\mathsf{PKE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ with a message space $\mathscr{M}$ is said to be correct if for any pair of secret and public keys $(\mathsf{sk}, \mathsf{pk})$ and for every message $m \in \mathscr{M}$,

$$\mathsf{PKE.Dec}(\mathsf{pp}_{\mathsf{PKE}}, \mathsf{PKE.Enc}(\mathsf{pp}_{\mathsf{PKE}}, m, \mathsf{pk}), \mathsf{sk}) \to m$$

where $\mathsf{pp}_{\mathsf{PKE}} \leftarrow \mathsf{PKE.Setup}(1^\lambda)$ and $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{PKE.KeyGen}(\mathsf{pp}_{\mathsf{PKE}})$.

Definition 2 (IND-CPA secure [18]). *A public key encryption scheme* $\mathsf{PKE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *with message space* $\mathscr{M}$ *is indistinguishable under chosen plaintext attack (IND-CPA) if for every PPT adversary* $\mathscr{A}$,

$$\Pr\left[\mu = \mu^* \left| \begin{array}{l} \mathsf{pp}_{\mathsf{PKE}} \leftarrow \mathsf{PKE.Setup}(1^\lambda), \\ (\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{PKE.KeyGen}(\mathsf{pp}_{\mathsf{PKE}}), \\ m_0, m_1 \leftarrow \mathscr{A}(\mathsf{pp}_{\mathsf{PKE}}, \mathscr{M}, \mathsf{pk}), \\ \mu \xleftarrow{\$} \{0,1\}, \\ \mathsf{ct}^* \leftarrow \mathsf{PKE.Enc}(\mathsf{pp}_{\mathsf{PKE}}, m_\mu, \mathsf{pk}), \\ \mu^* \leftarrow \mathscr{A}(\mathsf{pp}_{\mathsf{PKE}}, \mathsf{ct}^*, \mathsf{pk}) \end{array} \right.\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

Definition 3 (IND-CCA secure [18]). *A public key encryption scheme* $\mathsf{PKE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *with message space* $\mathscr{M}$ *is indistinguishable under chosen ciphertext attack (IND-CCA) if for every PPT adversary* $\mathscr{A}$,

$$\Pr\left[\mu = \mu^* \middle| \begin{array}{l} \mathsf{pp_{PKE}} \leftarrow \mathsf{PKE.Setup}(1^\lambda), \\ (\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{PKE.KeyGen}(\mathsf{pp_{PKE}}), \\ m_0, m_1 \leftarrow \mathscr{A}^{\mathscr{O}(\cdot)}(\mathsf{pp_{PKE}}, \mathscr{M}, \mathsf{pk}), \\ \mu \xleftarrow{\$} \{0,1\}, \\ \mathsf{ct}^* \leftarrow \mathsf{PKE.Enc}(\mathsf{pp_{PKE}}, m_\mu, \mathsf{pk}), \\ \mu^* \leftarrow \mathscr{A}^{\mathscr{O}(\cdot)}(\mathsf{pp_{PKE}}, \mathsf{ct}^*, \mathsf{pk}) \end{array}\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathscr{O}(\cdot)$ *is a decryption oracle that on input a ciphertext* $\mathsf{ct}' \neq \mathsf{ct}$*, outputs* $m' \leftarrow \mathsf{PKE.Dec}(\mathsf{pp_{PKE}}, \mathsf{ct}', \mathsf{sk})$ *or* $\perp$ *if the ciphertext* ct' *is invalid.*

2.2 Elliptic Curves and Isogenies

Suppose p is a prime number greater than or equal to 3 and K is a finite field of characteristic p i.e. $K = \mathbb{F}_{p^n}$ for some $n \geq 1$. Let $\overline{K}$ be the algebraic closure of K.

Definition 4 (Elliptic curve [31]). *An elliptic curve* E *over a finite field* K *with characteristic* $p > 3$ *is a projective non-singular curve with an affine equation* $y^2 = x^2 + Ax + B$ *and a unique point* $O \in E$ *called a point at infinity where* $A, B \in K$ *such that* $4A^3 + 27B^2 \neq 0$*. We denote the set of all points on the elliptic curve* E *as* $E(\overline{K}) = \{(x, y) \in \overline{K}^2 | y^2 = x^3 + Ax + B\} \cup \{O\}$*.*

Addition under the chord and tangent law for $P, Q \in E$. Let L be the chord through P and Q (L is the tangent to E at P if $P = Q$) which intersects E at a third point R. If L' is the line through R and O which intersects E at a third point R', then we define $R' = P + Q$.

Theorem 4 ([31]). *The elliptic curve* $E(\overline{K})$ *forms an abelian group under addition defined above by the chord and tangent law.*

The set of K-rational points on an elliptic curve E is the set $E(K) = \{(x, y) \in K^2 : y^2 = x^3 + Ax + B\} \cup \{O\}$ which forms a subgroup of $E(\overline{K})$.

Definition 5 (Isogeny [31]). *Let* E, E' *be two elliptic curves over a finite field* K*. An isogeny is a non-constant morphism* $\phi : E \to E'$ *over* $\overline{K}$ *satisfying* $\phi(O) = O$*. Two elliptic curves* E *and* E' *are called isogenous if there exists an isogeny* $\phi : E \to E'$ *between them with* $\phi(E) \neq \{O\}$*.*

Since an isogeny is a morphism between two elliptic curves then it is either constant or surjective. For an isogeny $\phi : E \to E'$, we define kernel of ϕ as $\mathsf{ker}(\phi) = \phi^{-1}(O)$.

Theorem 5 ([31]). *Let* $\phi : E \to E'$ *be an isogeny between two elliptic curves* E *and* E'*. Then* $\mathsf{ker}(\phi)$ *is a finite subgroup of* E

Given an isogeny $\phi : E \to E'$, we can obtain the usual injection of function fields $\phi^* : \overline{K}(E') \to \overline{K}(E)$ via the map $\phi^*(f) = f \circ \phi$ for $f \in \overline{K}(E')$, where $\overline{K}(E)$ and $\overline{K}(E')$ are the function fields corresponding to the elliptic curves E and E' respectively.

Definition 6 (Degree of an isogeny [31]). *Let $\phi : E \to E'$ be an isogeny between two elliptic curves E and E'. The degree of an isogeny $\phi : E \to E'$ is defined as the degree of the extension of the function field $\overline{K}(E)$ over the field $\phi^*(\overline{K}(E'))$ and is denoted by $\mathsf{deg}(\phi)$.*

Definition 7 (Endomorphism [31]). *An isogeny $\phi : E \to E$ is called endomorphism of an elliptic curve E.*

We denote $\mathsf{End}(E)$ as the ring of all endomorphisms of E defined over $\overline{K}$ and $\mathsf{End}_K(E)$ as the ring of all endomorphisms of E defined over K.

Definition 8 (Multiplication-by-m map [31]). *Let E be an elliptic curve and $m \in \mathbb{Z}$ with $m \neq 0$. The multiplication-by-m map, denoted by $[m] : E \to E$, is defined by $[m]P = \underbrace{P + P + \cdots + P}_{m\ terms}$, where $m > 0$ and $P \in E$. For $m < 0$, we define $[m]P = [-m](-P)$ and for $m = 0$, we define $[0]P = O$.*

Definition 9 (Frobenius map [31]). *Let E be an elliptic curve over a finite field K of characteristic p. The Frobenius map, denoted by $\pi : E \to E$, is defined by $\pi(x, y) = (x^p, y^p)$.*

The multiplication-by-m map and Frobenius map are examples of endomorphisms of an elliptic curve E.

Definition 10 (Torsion subgroup [31]). *Let E be an elliptic curve and $m \in \mathbb{N}$. The m-torsion subgroup of E, denoted by $E[m]$, is defined by the set $E[m] = \{P \in E | [m]P = O\}$.*

Definition 11 (Separable isogeny [31]). *An isogeny is said to be separable if the degree of the isogeny is the size of its kernel.*

Theorem 6 (Vélu's Formulae [35]). *Let $E : y^2 = x^3 + Ax + B$ be an elliptic curve defined over a finite field K and G be a finite subgroup of $E(\overline{K})$. Then there exists an elliptic curve E' and a separable isogeny $\phi : E \to E'$ with $\mathsf{ker}(\phi) = G$.*

Definition 12 (Supersingular elliptic curve [31]). *An elliptic curve E over a finite field K of characteristic p is said to be supersingular if $E[p^r] = \{O\}$ for all $r \geq 1$. Otherwise, it is said to be an ordinary elliptic curve.*

Remark 1. In general, the $\mathbb{F}_p$-endomorphism ring of a supersingular elliptic curve E is isomorphic to either $\mathbb{Z}[\pi]$ or $\mathbb{Z}[\frac{1+\pi}{2}]$ [9]. For an ordinary elliptic curve E, it holds that $\mathsf{End}(E) = \mathsf{End}_{\mathbb{F}_p}(E)$ whereas for a supersingular elliptic curve E, it holds that $\mathsf{End}_{\mathbb{F}_p}(E) \subsetneq \mathsf{End}(E)$. For a supersingular elliptic curve E, it is known that its endomorphism ring $\mathsf{End}(E)$ is isomorphic to a maximal order of a quaternion algebra and $\mathsf{End}_{\mathbb{F}_p}(E)$ is isomorphic to an order of an imaginary quadratic field $\mathbb{Q}(\sqrt{-p})$. An order is a subring of a field K which is a free $\mathbb{Z}$-module of rank 2.

Proposition 1 ([31]). *Let E be an elliptic curve over a finite field $\mathbb{F}_p$ and $t = p + 1 - |E(\mathbb{F}_p)|$. Then the Frobenius map π satisfies $\pi^2 - t\pi + p = 0$ where t is the trace of the Frobenius map.*

Proposition 2 ([31]). *Let $p \geq 5$ be a prime. Then an elliptic curve E is supersingular over $\mathbb{F}_p$ if and only if the trace of the Frobenius map is 0, i.e., $|E(\mathbb{F}_p)| = p + 1$.*

Definition 13 (Montgomery curve [24]). *A Montgomery curve over $\mathbb{F}_p$ is defined by the affine equation $By^2 = x^3 + Ax^2 + x$ for some $A, B \in \mathbb{F}_p$ satisfying $B(A^2 - 4) \neq 0$.*

2.3 Class Group Action

Let K be a quadratic number field i.e. an extension field of $\mathbb{Q}$ with degree 2 and $\mathfrak{O} \subseteq K$ be an order. A quadratic number field K is called an imaginary quadratic number field if it is of the form $\mathbb{Q}(\sqrt{-d})$ where d is a square-free integer.

Definition 14 (Fractional Ideal [33]). *A fractional ideal of $\mathfrak{O}$ is a submodule of K of the form $\alpha\mathfrak{a}$ where $\alpha \in K^*$ and $\mathfrak{a}$ is an $\mathfrak{O}$-ideal. A fractional $\mathfrak{O}$-ideal $\mathfrak{a}$ is called invertible if there exists a fractional $\mathfrak{O}$-ideal $\mathfrak{b}$ such that $\mathfrak{ab} = \mathfrak{O}$.*

We denote by $I(\mathfrak{O}) = \{\mathfrak{a} \in \mathfrak{O} | \exists \mathfrak{b} \in \mathfrak{O} \text{ such that } \mathfrak{ab} = \mathfrak{O}\}$ the set of all invertible fractional ideals of $\mathfrak{O}$ and by $P(\mathfrak{O}) = \{\mathfrak{a} | \mathfrak{a} = \alpha\mathfrak{O} \text{ for some } \alpha \in K\}$ the set of all invertible principal fractional ideals of $\mathfrak{O}$.

Proposition 3 ([33]). *The set $I(\mathfrak{O})$ forms an abelian group under ideal multiplication and $P(\mathfrak{O})$ forms a normal subgroup of $I(\mathfrak{O})$.*

Definition 15 (Ideal Class group [33]). *The ideal class group $\mathsf{cl}(\mathfrak{O})$ of an order $\mathfrak{O}$ is defined by the quotient group $\mathsf{cl}(\mathfrak{O}) = I(\mathfrak{O})/P(\mathfrak{O})$ and given a fractional ideal $\mathfrak{a} \in I(\mathfrak{O})$, we denote $[\mathfrak{a}]$ as its equivalence class in $\mathsf{cl}(\mathfrak{O})$.*

Theorem 7 ([30]). *The class group $\mathsf{cl}(\mathfrak{O})$ forms a finite abelian group.*

Class group action on supersingular elliptic curves over $\mathbb{F}_p$. Throughout this paper, we will use the endomorphism ring over $\mathbb{F}_p$ and denote it as $\mathsf{End}_{\mathbb{F}_p}(E) = \mathcal{O}$. We denote by $\mathscr{E}_p(\mathcal{O})$ the set of all supersingular elliptic curves defined over $\mathbb{F}_p$ with the $\mathbb{F}_p$-endomorphism ring $\mathcal{O}$. Given an ideal $\mathfrak{a} \subseteq \mathcal{O}$, we define a corresponding finite subgroup $E[\mathfrak{a}] \subseteq E$ as $E[\mathfrak{a}] = \cap_{a \in \mathfrak{a}} \mathsf{ker}(a)$. Since $E[\mathfrak{a}]$ is a finite subgroup, we can compute an isogeny $\phi_{\mathfrak{a}}$ from E to a new isogeny $E' = E/E[\mathfrak{a}]$ by applying Vélu's formula given in Theorem 6. For simplicity, we will denote $E' = E/E[\mathfrak{a}]$ by $[\mathfrak{a}]E$. This defines a group action of the ideal class group $\mathsf{cl}(\mathcal{O})$ on $\mathscr{E}_p(\mathcal{O})$ via the map

$$\begin{aligned} \star : \mathsf{cl}(\mathcal{O}) \times \mathscr{E}_p(\mathcal{O}) &\to \mathscr{E}_p(\mathcal{O}) \\ ([\mathfrak{a}], E) &\mapsto [\mathfrak{a}] \star E = [\mathfrak{a}]E \end{aligned}$$

where $[\mathfrak{a}] \in \mathsf{cl}(\mathcal{O})$ and $E \in \mathscr{E}_p(\mathcal{O})$.

Theorem 8 ([36]). *Let $\mathcal{O} = \mathsf{End}_{\mathbb{F}_p}(E)$ be an order in an imaginary quadratic number field such that $\mathscr{E}_p(\mathcal{O})$ is non-empty. Then the ideal class group $\mathsf{cl}(\mathcal{O})$ acts freely and transitively on the set $\mathscr{E}_p(\mathcal{O})$ via the map $\star$ defined above.*

Proposition 4 ([5]). *Let $p > 3$ be a prime that satisfies $p \equiv 3 \pmod 4$ and E be a supersingular elliptic curve over $\mathbb{F}_p$. If $\mathsf{End}_{\mathbb{F}_p}(E) \cong \mathbb{Z}[\pi]$, then there exists an element $A \in \mathbb{F}_p$ such that E is $\mathbb{F}_p$-isomorphic to the curve $E_A : y^2 = x^3+Ax^2+x$. Moreover, if such an A exists, then it is unique.*

Assumption 1 (CSSICDH [22]). *The Commutative Supersingular Isogeny Computational Diffie-Hellman (CSSICDH) assumption holds if for any PPT adversary $\mathscr{A}$,*

$$\Pr\left[E = [\mathfrak{b}][\mathfrak{a}]E_0 | [\mathfrak{a}], [\mathfrak{b}] \leftarrow \mathsf{cl}(\mathcal{O}), E \leftarrow \mathscr{A}(E_0, [\mathfrak{a}]E_0, [\mathfrak{b}]E_0)\right] < \mathsf{negl}(\lambda)$$

Assumption 2 (CSSIDDH [22]). *The Commutative Supersingular Isogeny Decisional Diffie-Hellman (CSSIDDH) assumption holds if for any PPT adversary $\mathscr{A}$,*

$$\left| \Pr\left[\mu = \mu^* \middle| \begin{array}{l} [\mathfrak{a}], [\mathfrak{b}], [\mathfrak{c}] \leftarrow \mathsf{cl}(\mathcal{O}), \mu \xleftarrow{\$} \{0,1\}, \\ F_0 = [\mathfrak{b}][\mathfrak{a}]E_0, F_1 = [\mathfrak{c}]E_0, \\ \mu^* \leftarrow \mathscr{A}(E_0, [\mathfrak{a}]E_0, [\mathfrak{b}]E_0, F_b) \end{array} \right] - \frac{1}{2} \right| < \mathsf{negl}(\lambda)$$

2.4 Pohlig-Hellman Algorithm for Montgomery Curves

The Pohlig-Hellman algorithm [26] is a method used to solve the discrete logarithm problem in certain finite groups. The algorithm is based on the observation that if the order of the group has small prime factors, then the discrete logarithm problem can be solved in polynomial time. More formally, if a cyclic group G has order $p_1^{k_1} \cdots p_t^{k_t}$ where p_i's are small primes and $k_i \geq 0$ are integers for $i \in \{1, \ldots, t\}$ then the discrete logarithm problem in G can be solved efficiently. For our construction, we focus only on the cyclic group $\mathbb{Z}/2^r\mathbb{Z}$. Let $\mu \in \mathbb{Z}/2^r\mathbb{Z}$ with $\mu = \sum_{i=0}^{r-1} \mu_i 2^i$ where $\mu_i \in \{0, 1\}$ and $\mathbb{Z}/2^r\mathbb{Z} = \langle P \rangle$. Given two points P and μP, one can compute μ in the following way:

- **Computation of** μ_0
  - Compute $2^{r-1} \cdot \mu P$ using given μP
  - Set $\mu_0 = 0$ if $2^{r-1} \cdot \mu P = 0$ and $\mu_0 = 1$ if $2^{r-1} \cdot \mu P \neq 0$.
- **Computation of** μ_j
  - After computing $\mu_1, \mu_2, \ldots \mu_{j-1}$ recursively by calculating $2^{r-j-1} \cdot \mu^{(j-1)} P$ where $\mu^{(k)} = \mu - \sum_{i=0}^{k-1} \mu_i 2^i$ for $k = 1, 2 \ldots, j-1$, compute $\mu^{(j)} = \mu - \sum_{i=0}^{j-1} \mu_i 2^i$.
  - Set $\mu_j = 0$ if $2^{r-j-1} \cdot \mu^{(j)} P = 0$ and $\mu_j = 1$ if $2^{r-j-1} \cdot \mu^{(j)} P \neq 0$.

Using this idea, Moriya et al. [22] describe the Pohlig-Hellman algorithm for Montgomery curves in their construction of SiGamal. On input a Montgomery curve $E_A : y^2 = x^3 + Ax^2 + x$ over $\mathbb{F}_p$, x-cordinates of points $P, Q \in E_A$ of order 2^r with $Q \in \langle P \rangle$, the Pohlig-Hellman algorithm outputs M where M is either μ or $2^r - \mu$ satisfying $P = MQ$. With these notations, we summarise the Pohlig-Hellman algorithm for Montgomery curves in Algorithm 1.

Algorithm 1: $\mathsf{Pohlig\text{-}Hellman}(E_A : y^2 = x^3 + Ax^2 + x, x(P), x(Q)) \rightarrow M$ where $P, Q \in E_A$ of order 2^r with $Q \in \langle P \rangle$ and M is either μ or $2^r - \mu$ satisfying $P = MQ$

Set $x(P_0) \leftarrow x(P)$ and $x(Q_0) \leftarrow x(Q)$;
for $(i = 1; i \leq r - 2; i++)$ **do**
    $x(P_i) \leftarrow x(2P_{i-1})$, $x(Q_i) \leftarrow x(2Q_{i-1})$;
end for
$M \leftarrow 1$;
for $(i = 2; i \leq r - 1; i++)$ **do**
    $x(R) \leftarrow x(MQ_{r-i})$;
    if $(x(P_{r-i}) \neq x(R))$ **then**
        $M \leftarrow M + 2^i$;
    end if
end for
return M

2.5 SimS (Simplification of SiGamal): An IND-CCA Secure Encryption Scheme from Isogenies

In Asiacrypt 2020, Moriya et al. [22] proposed an IND-CPA secure encryption scheme, SiGamal based on *commutative supersingular isogney-based Diffie-Hellman* (CSIDH) key exchange of Castryk et al. [6]. Later, Fouotsa et al. [12] introduced an IND-CCA secure encryption scheme SimS (Simplification of SiGamal) from CSIDH in PQCrypto 2021. The scheme SimS used a prime p of the form $p = 2^r \ell_1 \ell_2 \cdots \ell_n - 1$ where $\ell_1, \ell_2, \ldots, \ell_n$ are small odd primes and $\lambda + 2 \leq r \leq \frac{1}{2} \log p$ instead of $p = 4\ell_1 \ell_2 \cdots \ell_n - 1$ used in CSIDH [6]. Since $p \equiv 3 \pmod 4$, Proposition 4 can be used to compute the group action. The scheme SimS uses the deterministic procedure FindPoint [12] given in Algorithm 2 to find a point P_{E_A} of order 2^r on the Montgomery curve $E_A : y^2 = x^3 + Ax^2 + x$ defined over $\mathbb{F}_p$. This algorithm relies on the following result.

Theorem 9 ([12]). *Let $p = 2^r \ell_1 \ell_2 \cdots \ell_n - 1$ be a prime such that $p \equiv 3 \pmod 4$ and let E be a supersingular Montgomery curve defined over $\mathbb{F}_p$ satisfying $\mathsf{End}_{\mathbb{F}_p}(E) \cong \mathbb{Z}[\pi]$. Let $P \in E(\mathbb{F}_p)$ such that $x(P) \neq 0$. If $x(P)$ is not a square in $\mathbb{F}_p^*$ then $[\ell_1 \ell_2 \cdots \ell_n]P$ is a point of order 2^r.*

The scheme SimS uses the following notion of randomizing function defined in Definition 16 to guarantee IND-CCA security of SimS.

Definition 16 (Randomizing function [12]). *A map $f_E : \mathbb{F}_p \rightarrow \mathbb{F}_p$, indexed by a supersingular curve E, is called randomizing function if it satisfies the following properties:*

- *($\mathscr{P}$1) The map f_E is bijective. The map f_E and its inverse $g_E = f_E^{-1}$ are efficiently computable when E is given.*
- *($\mathscr{P}$2) For every element $x \in \mathbb{F}_p$, a PPT adversary $\mathscr{A}$ having no access to x and E, cannot distinguish between $f_E(x)$ and a random element of $\mathbb{F}_p$.*

Algorithm 2: $\mathsf{FindPoint}(p = 2^r \ell_1 \cdots \ell_n - 1, r, \ell_1, \ell_2, \ldots, \ell_n, \mathsf{MC}(E_A) = A \in \mathbb{F}_p) \to P_{E_A} \in E_A$, where P_{E_A} is a point of order 2^r

Set $x \leftarrow -2$;
while ($x^3 + Ax^2 + x$ *is not a square in* $\mathbb{F}_p$ *and* $-x \leq \ell_{n-1} + 1$) **do**
    Set $x \leftarrow x - 1$;
end while
if ($-x \leq \ell_{n-1} + 1$) **then**
    Set $P = (x, y) \in E_A : y^2 = x^3 + Ax^2 + x$ and $P_{E_A} = [\ell_1 \cdots \ell_n]P$;
    return P_{E_A}
else
    return $\perp$
end if

($\mathscr{P}3$) *For every element $x \in \mathbb{F}_p$, for every non-identical rational function $R \in \mathbb{F}_p(X)$, a PPT adversary $\mathscr{A}$ having no access to x and E, cannot compute $f_E(R(x))$ from $f_E(x)$.*

An instantiation of the randomizing function indexed by a supersingular elliptic curve is described in Figure 1.

Instantiation

– Define a map $f_E : \mathbb{F}_p \to \mathbb{F}_p$ by $f_E(x) = x'$ satisfying $\mathsf{bin}(x') = \mathsf{bin}(x) \oplus \mathsf{bin}(A_E)$ where $A_E = \mathsf{MC}(E)$ is the Montgomery coefficient of the elliptic curve E and $\mathsf{bin}(x)$ is the binary representation of an element $x \in \mathbb{F}_p$.
– The function f_E is a randomizing function indexed by a supersingular elliptic curve E.
– The inverse map g_E of f_E is $g_E(x') = x$ satisfying $\mathsf{bin}(x) = \mathsf{bin}(x') \oplus \mathsf{bin}(A_E)$.
– As $\mathsf{bin}(f_E(x)) \oplus \mathsf{bin}(A_E) = \mathsf{bin}(x) \oplus \mathsf{bin}(A_E) \oplus \mathsf{bin}(A_E) = \mathsf{bin}(x)$, we get $g_E(f_E(x)) = x$.

Fig. 1. Instantiation of Randomizing function [12]

With the notations above, we summarise the encryption scheme $\mathsf{SimS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ in Algorithms 3, 4, 5, 6 respectively. The Setup algorithm is run by a trusted authority who generates the public parameter $\mathsf{pp}_{\mathsf{SimS}}$ and makes it public. The algorithm KeyGen is run by a user who generates its secret-public key pair $(\mathsf{sk}, \mathsf{pk})$ using $\mathsf{pp}_{\mathsf{SimS}}$. It makes pk public and keeps sk secret to itself. The encryption algorithm Enc is run by a sender who uses the receiver's public key pk to encrypt a message $m \in \mathbb{Z}_{2^{r-2}}$ using $\mathsf{pp}_{\mathsf{SimS}}$ and generates a ciphertext $\mathsf{ct}_{\mathsf{SimS}}$. The receiver with its secret key sk can recover the message by running the decryption algorithm Dec using $\mathsf{pp}_{\mathsf{SimS}}$ and the ciphertext $\mathsf{ct}_{\mathsf{SimS}}$. Both the encryption and decryption algorithm calls Algorithm 2 once to find a point of order 2^r on an elliptic curve. The decryption algorithm additionally makes a call of the Pohlig-Hellman algorithm presented in Algorithm 1.

Algorithm 3: $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}_{\mathsf{SimS}}$

---

1 Choose n small distinct odd primes $\ell_1, \ell_2, \ldots, \ell_n$ such that $p = 2^r \ell_1 \ell_2 \cdots \ell_n - 1$ is a prime number where $\lambda + 2 \leq r \leq \frac{1}{2} \log p$;
2 Set the base elliptic curve $E_0 : y^2 = x^3 + x \in \mathscr{E}_p(\mathscr{O})$;
3 Choose an integer s such that $(2s+1)^n \geq |\mathsf{cl}(\mathscr{O})|$;
4 Choose a family of randomizing functions $\{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}$ indexed by supersingular elliptic curves where $f_E(x) = x'$ satisfying $\mathsf{bin}(x') = \mathsf{bin}(x) \oplus \mathsf{bin}(A_E)$ for a supersingular Montgomery elliptic curve E with Montgomery coefficient $\mathsf{MC}(E) = A_E$ and inverse g_E of f_E is defined by $g_E(x') = \mathsf{bin}(x') \oplus \mathsf{bin}(A_E)$;
5 **return** the public parameter $\mathsf{pp}_{\mathsf{SimS}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E \in \mathscr{E}_p(\mathscr{O})})$

Algorithm 4: $\mathsf{KeyGen}(\mathsf{pp}_{\mathsf{SimS}}) \to (\mathsf{sk}, \mathsf{pk})$

---

1 Sample $(a_1, \ldots, a_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$ and define an ideal class $[\mathfrak{a}] = \prod_{i=1}^n [\mathfrak{l}_i^{a_i}] \in \mathsf{cl}(\mathscr{O})$ where $\mathfrak{l}_i = \langle \ell_i, \pi - 1 \rangle$ for $i = 1, 2 \ldots, n$ and π is the Frobenius map;
2 Compute the class group action $E_1 = [\mathfrak{a}]E_0$ and set the public key $\mathsf{pk} = E_1$ and the secret key $\mathsf{sk} = [\mathfrak{a}]$;
3 **return** the secret-public key pair $(\mathsf{sk}, \mathsf{pk})$

Algorithm 5: $\mathsf{Enc}(\mathsf{pp}_{\mathsf{SimS}}, m \in \mathbb{Z}_{2^{r-2}}, \mathsf{pk}) \to \mathsf{ct}_{\mathsf{SimS}}$

---

1 Set $m' = 2m + 1$;
2 Sample $(b_1, \ldots, b_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$ and define an ideal class $[\mathfrak{b}] = \prod_{i=1}^n [\mathfrak{l}_i^{e'_i}] \in \mathsf{cl}(\mathscr{O})$;
3 Compute $E_3 = [\mathfrak{b}]E_0, E_4 = [\mathfrak{b}]E_1$;
4 Run the procedure $\mathsf{FindPoint}(p, r, \ell_1, \ldots, \ell_n, \mathsf{MC}(E_4))$ given in Algorithm 2 to find a point $P_{E_4} \in E_4$ of order 2^r;
5 Compute $P_4 = [m']P_{E_4}$ and $x' = f_{E_4}(x(P_4))$;
6 **return** the ciphertext $\mathsf{ct}_{\mathsf{SimS}} = (E_3, x')$

Correctness: Note that f_{E_4} is bijective and its inverse $g_{E_4} = f_{E_4}^{-1}$ is efficiently computable by the receiver and the receiver can compute $E_4 = [\mathfrak{a}]E_3$. The

Algorithm 6: $\mathsf{Dec}(\mathsf{pp}_{\mathsf{SimS}}, \mathsf{ct}_{\mathsf{SimS}} = (E_3 = [\mathfrak{b}]E_0, x'), \mathsf{sk} = [\mathfrak{a}]) \to m \vee \perp$

if (E_3 *supersingular*) **then**
    Compute $E_4 = [\mathfrak{a}]E_3$;
    Run the procedure $\mathsf{FindPoint}(p, r, \ell_1, \ldots, \ell_n, \mathsf{MC}(E_4))$ given in Algorithm 2 to find a point $P_{E_4} \in E_4$ of order 2^r;
    Compute $g_{E_4}(x')$ by extracting x' from $\mathsf{ct}_{\mathsf{SimS}}$;
    if ($g_{E_4}(x')$ *is not the x-coordinate of a* 2^r*-torsion point on* E_4) **then**
        return $\perp$
    else
        Find $M \in \mathbb{Z}_{2^r}^{\times}$ by applying the procedure $\mathsf{Pohlig\text{-}Hellman}(E_4, x(P_{E_4}), x(P_4) = g_{E_4}(x'))$ given in Algorithm 1;
    end if
    if ($2^{r-1} < M$) **then**
        Set $M' = 2^r - M$;
    else
        Set $M' = M$;
    end if
    return $m = \frac{M'-1}{2}$
else
    return $\perp$
end if

Montgomery coefficients of the curves $[\mathfrak{a}][\mathfrak{b}]E_0$ (corresponding to the receiver) and $[\mathfrak{b}][\mathfrak{a}]E_0$ (corresponding to the sender) are equal due to the commutativity of $\mathsf{Cl}(\mathscr{O})$ (Theorem 7). Thus the sender and the receiver both get the same point P_{E_4}. As the points P_{E_4} and $P_4 = [2m+1]P_{E_4}$ have order 2^r, the Pohlig-Hellman algorithm can be invoked on the Montgomery curve E_4, the x-coordinates $x(P_4)$ and $x(P_{E_4})$ of the respective points P_4, P_{E_4} of order 2^r each with $P_{E_4} = \langle P_4 \rangle$ to recover $M = \pm(2m+1) \pmod{2^r}$. Since $m \in \mathbb{Z}_{2^{r-2}}$, we have $2m+1 < 2^{r-1}$. The receiver changes M to $2^r - M$ if $2^{r-1} < M$ and finally computes the plaintext $(M-1)/2 = m$.

The scheme SimS achieves IND-CCA security under the following assumption.

Assumption 3 (CSSIKOE-I [12])**.** *Let λ be a security parameter and $p = 2^r\ell_1 \cdots \ell_n - 1$ be a prime such that $\lambda + 2 \le r \le \frac{1}{2}\log p$ where $\ell_1, \ldots, \ell_n$ are small odd primes. Let $(a_1, \ldots, a_n)$ and $(b_1, \ldots, b_n)$ are uniformly sampled from $\{-s, \ldots, s\}^n$. Define $[\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}]$, $[\mathfrak{b}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{b_i}] \in \mathsf{cl}(\mathscr{O})$. Let $\{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}$ be a family of randomizing functions indexed by supersingular elliptic curves and $E_0 : y^2 = x^3 + x$ over $\mathbb{F}_p$ be the base supersingular elliptic curve. The Commutative Supersingular Isogeny Knowledge of Exponent I (CSSIKOE-I) assumption is stats that for every PPT adversary $\mathscr{A}$, if $\mathscr{A}(E_0, [\mathfrak{a}]E_0, ([\mathfrak{b}]E_0, f_{[\mathfrak{a}][\mathfrak{b}]E_0}(x(P)))) \to ([\mathfrak{b}']E_0, f_{[\mathfrak{a}][\mathfrak{b}']E_0}(x(P'))) \neq ([\mathfrak{b}]E_0, f_{[\mathfrak{a}][\mathfrak{b}]E_0}(x(P)))$ where $P \in [\mathfrak{a}][\mathfrak{b}]E_0$, $P' \in [\mathfrak{a}][\mathfrak{b}']E_0$ are points of order 2^r each and $[\mathfrak{b}'] \in \mathsf{cl}(\mathscr{O})$, then there exists a PPT ad-*

versary $\mathscr{A}'$ such that $\mathscr{A}'(E_0, [\mathfrak{a}]E_0, ([\mathfrak{b}]E_0, f_{[\mathfrak{a}][\mathfrak{b}]E_0}(x(P)))) \to ([\mathfrak{b}'], [\mathfrak{b}']E_0, f_{[\mathfrak{a}][\mathfrak{b}']E_0}$ $(x(P)))$.

Theorem 10 ([12]). *If Assumption 2 (CSSIDDH), Assumption 3(CSSIKOE-I) hold and the family of randomizing functions $\{f_E\}_{E\in\mathscr{E}_p(\mathcal{O})}$ indexed by supersingular elliptic curves satisfies the property ($\mathscr{P}2$) as per Definition 16, then* SimS *is IND-CCA secure.*

2.6 Bidirectional Proxy Re-encryption

In this section, we review the definition and the security notions of a bidirectional proxy re-encryption scheme following Canetti et al. [4]. Bidirectional proxy re-encryption allows a semi-trusted proxy server to convert a ciphertext ct_i encrypted with the delegator's public key pk_i into a different ciphertext ct_j under the delegatee's public key pk_j without disclosing the underlying secret messages to the proxy server.

Definition 17 (Bidirectional Proxy Re-encryption [4]). *A bidirectional proxy re-encryption scheme* bPRE *with message space $\mathscr{M}$ is a tuple of five algorithms* $\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}$, ReEnc *and a three-party protocol* $\mathsf{ReKeyGen}$ *among a delegator, a delegate and a semi-trusted proxy server satisfying the following requirements:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp_{bPRE}}$. *On input a security parameter λ, a trusted authority runs this PPT algorithm and outputs a public parameter $\mathsf{pp_{bPRE}}$ which is made publicly available.*
- $\mathsf{KeyGen}(\mathsf{pp_{bPRE}}) \to (\mathsf{sk}, \mathsf{pk})$. *On input the public parameter $\mathsf{pp_{bPRE}}$, a user runs this PPT algorithm and outputs a public key pk and a corresponding secret key sk. The public key pk is made public and the secret key sk is kept secret to the user.*
- $\mathsf{Enc}(\mathsf{pp_{bPRE}}, m, \mathsf{pk}) \to \mathsf{ct}$. *On input the public parameter $\mathsf{pp_{bPRE}}$, receiver's public key pk and a message $m \in \mathscr{M}$, a sender runs this PPT algorithm to produce a ciphertext ct which is sent over a public channel to the receiver.*
- $\mathsf{Dec}(\mathsf{pp_{bPRE}}, \mathsf{ct}, \mathsf{sk}) \to m/\perp$. *On input the public parameter $\mathsf{pp_{bPRE}}$, receiver's secret key sk and a ciphertext ct, a receiver runs this deterministic algorithm and recovers either a message $m \in \mathscr{M}$ or an error symbol $\perp$, indicating decryption failure.*
- $\mathsf{ReKeyGen}(\mathsf{pp_{bPRE}}, \mathsf{sk}_i, \mathsf{sk}_j) \to \mathsf{rk}_{i\leftrightarrow j}$. *On input the public parameter $\mathsf{pp_{bPRE}}$, a semi-trusted proxy server, a delegator with its secret key sk_i and a delegatee with its secret key sk_j together run this PPT algorithm using a three-party protocol and produce a bidirectional re-encryption key $\mathsf{rk}_{i\leftrightarrow j}$.*
- $\mathsf{ReEnc}(\mathsf{pp_{bPRE}}, \mathsf{rk}_{i\leftrightarrow j}, \mathsf{ct}_i) \to \mathsf{ct}_j$. *On input the public parameter $\mathsf{pp_{bPRE}}$, a re-encryption key $\mathsf{rk}_{i\leftrightarrow j}$ and a ciphertext ct_i encrypted under delegator's public key pk_i from a delegator U_i, a semi-trusted proxy server runs this PPT algorithm and generates a ciphertext ct_j encrypted under delegatee's public key.*

Correctness. A bidirectional proxy re-encryption $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$ is said to be correct if for all λ, all $m \in \mathscr{M}$, all $\mathsf{pp}_{\mathsf{bPRE}} \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$ and all $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$, it holds that

$$\mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m, \mathsf{pk}), \mathsf{sk}) \to m$$

and for all $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$ and $(\mathsf{sk}_j, \mathsf{pk}_j) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp})$, it holds that

$$\mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{bPRE.ReEnc}(\mathsf{bPRE.ReKeyGen}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{sk}_i, \mathsf{sk}_j), \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m, \mathsf{pk}_i)), \mathsf{sk}_j) \to m.$$

Remark 2. It is required that the re-encryption key must be generated using a protocol involving the semi-trusted proxy server and two secret key holders where at most one of the three parties may be corrupted, otherwise we have no longer security requirements [4].

Definition 18 (Multi-hop bidirectional proxy re-encryption [23]). *A bidirectional proxy re-encryption scheme* $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$ *with message space* $\mathscr{M}$ *is called multi-hop if for any sequence of secret key and private key pairs* $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$ *with* $1 \leq i \leq N$*, all re-encryption keys* $\mathsf{rk}_{j\leftrightarrow j+1} \leftarrow \mathsf{bPRE.ReKeyGen}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{sk}_j, \mathsf{sk}_{j+1})$ *with* $j < N$ *and all messages* $m \in \mathscr{M}$ *it holds that*

$$\mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{bPRE.ReEnc}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{rk}_{N-1\leftrightarrow N}, \ldots, \mathsf{bPRE.ReEnc}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{rk}_{1\leftrightarrow 2}, \mathsf{ct}_1)), \mathsf{sk}_N) \to m$$

where $\mathsf{ct}_1 \leftarrow \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m, \mathsf{pk}_1)$ *and* $\mathsf{pp}_{\mathsf{bPRE}} \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$.

Security definitions. Following Canetti et al. [4] and Nunez et al. [23], we describe below the chosen plaintext security (CPA) for the bidirectional proxy re-encryption scheme $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$ which is modeled as a security game between a challenger $\mathscr{C}$ and an adversary $\mathscr{A}$.

Definition 19 (CPA security of bidirectional proxy re-encryption [4], [23]). *A bidirectional proxy re-encryption scheme* $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$ *is CPA secure if the advantage* $\mathsf{Adv}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda)$ *of any PPT adversary* $\mathscr{A}$*, defined as*

$$\mathsf{Adv}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda) = \left|\Pr[\mathsf{Exp}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda) = 1] - \frac{1}{2}\right|$$

is negligible in the security parameter λ *where* $\mathsf{Exp}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda)$ *is the security game between the adversary* $\mathscr{A}$ *and a challenger* $\mathscr{C}$ *described below where* Γ_U *and* Γ_C *are two lists (initially empty) maintained by the challenger* $\mathscr{C}$ *for keeping records of the respective indices of uncorrupted and corrupted users.*

$\mathsf{Exp}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda)$

Phase 0:

- ***Setup:*** *On input a security parameter* λ*, the challenger* $\mathscr{C}$ *generates a public parameter* $\mathsf{pp}_{\mathsf{bPRE}} \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$ *and sends* $\mathsf{pp}_{\mathsf{bPRE}}$ *to* $\mathscr{A}$*.*

Phase 1: The adversary can invoke the following oracles polynomial times in any order.

- ***Uncorrupted key generation*** $\mathscr{O}_{Uncorrupt}$*: On input an user index* i *with* $i \notin \Gamma_U \cup \Gamma_C$*, the challenger* $\mathscr{C}$ *generates a key pair* $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$*, adds* i *in the list of indices of uncorrupted users* Γ_U *(initially empty), sends* pk_i *to* $\mathscr{A}$ *and keeps* sk_i *secret to itself. Here* Γ_C *(initially empty) is the list of indices of corrupted users.*
- ***Corrupted key generation*** $\mathscr{O}_{Corrupt}$*: On input an index* i *with* $i \notin \Gamma_U \cup \Gamma_C$*, the challenger* $\mathscr{C}$ *generates a key pair* $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$*, adds* i *in the list of indices of corrupted users* Γ_C *(initially empty) and sends* $(\mathsf{sk}_i, \mathsf{pk}_i)$ *to* $\mathscr{A}$*.*

Phase 2:

- ***Re-encryption key generation*** $\mathscr{O}_{Rkgen}$*: The adversary* $\mathscr{A}$ *can issue polynomially many queries to this oracle. On query* (i, j) *from the adversary* $\mathscr{A}$ *where* $i \neq j$*, the challenger* $\mathscr{C}$ *returns the re-encryption key* $\mathsf{rk}_{i \leftrightarrow j} \leftarrow \mathsf{bPRE.ReKeyGen}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{sk}_i, \mathsf{sk}_j)$ *to* $\mathscr{A}$ *where* sk_i *and* sk_j *are the secret keys corresponding to* pk_i *and* pk_j *respectively. Here it is required that either both* $i, j \in \Gamma_U$ *or both* $i, j \in \Gamma_C$
- ***Challenge phase:*** *In this phase the challenger generates the challenge public key* pk_{i^*} *of the target user* i^**, adds* i^* *in* Γ_U *and sends* pk_{i^*} *to the adversary* $\mathscr{A}$*. On query the challenge message pair* $(m_0, m_1) \in \mathscr{M}^2$ *from* $\mathscr{A}$*, the challenger* $\mathscr{C}$ *randomly chooses a bit* $\mu \xleftarrow{\$} \{0,1\}$ *and gives the challenge ciphertext* $\mathsf{ct}^* \leftarrow \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m_\mu, \mathsf{pk}_{i^*})$ *to* $\mathscr{A}$*.*

Phase 3:

- ***Decision phase:*** *The adversary* $\mathscr{A}$ *eventually guess a bit* $\mu^* \in \{0,1\}$ *to the challenger* $\mathscr{C}$ *who in turn returns* β *where*

$$\beta = \begin{cases} 1 & \text{if } \mu = \mu^* \\ 0 & \text{if } \mu \neq \mu^* \end{cases}$$

The output of the game $\mathsf{Exp}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda)$ *is* β*.*

3 Our Constructions

3.1 bPRE: Our Construction for Bidirectional proxy Re-encryption scheme

We describe below our bidirectional proxy re-encryption scheme bPRE consists of five PPT algorithms $\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReEnc}$ together with a three-party protocol $\mathsf{ReKeyGen}$ among a delegator, a delegate and a semi-trusted proxy server satisfying the following requirements.

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}_{\mathsf{bPRE}}$. On input a security parameter λ, a trusted authority runs this algorithm and generates a public parameter $\mathsf{pp}_{\mathsf{bPRE}}$ as follows.
  - (i) The trusted authority chooses n small distinct odd primes $\ell_1, \ell_2, \ldots, \ell_n$ such that $p = 2^r \ell_1 \ell_2 \cdots \ell_n - 1$ is a prime number where $\lambda + 2 \leq r \leq \frac{1}{2}\log p$.
  - (ii) It sets the base elliptic curve $E_0 : y^2 = x^3 + x \in \mathscr{E}_p(\mathscr{O})$ and chooses an integer s such that $(2s+1)^n \geq |\mathsf{cl}(\mathscr{O})|$.
  - (iii) It chooses two families of randomizing functions $\{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E \in \mathscr{E}_p(\mathscr{O})}$ indexed by supersingular elliptic curves where $g_E = f_E^{-1}$ using the instantiation given in Figure 1 satisfying the properties $(\mathscr{P}1) - (\mathscr{P}2)$ as in Definition 16.
  - (iv) Finally, it sets the public parameter

    $$\mathsf{pp}_{\mathsf{bPRE}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E \in \mathscr{E}_p(\mathscr{O})})$$

    and makes it public.
- $\mathsf{KeyGen}(\mathsf{pp}_{\mathsf{bPRE}}) \to (\mathsf{sk}, \mathsf{pk})$. On input the public parameter $\mathsf{pp}_{\mathsf{bPRE}}$, a user runs this algorithm to generate its public key and secret key pair $(\mathsf{sk}, \mathsf{pk})$ by executing the steps below.
  - (i) The user samples $(a_1, \ldots, a_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$, computes the ideal class $[\mathfrak{a}] = \prod_{i=1}^n [\mathfrak{l}_i^{a_i}] \in \mathsf{cl}(\mathscr{O})$ and sets the secret key $\mathsf{sk} = (a_1, \ldots, a_n)$. Here $\mathfrak{l}_i = \langle \ell_i, \pi - 1 \rangle$ for $i = 1, 2 \ldots, n$ and π is the Frobenius map.
  - (ii) It computes the class group action $E_A = [\mathfrak{a}]E_0$ and sets the public key $\mathsf{pk} = E_A$.
  - (iii) Finally, it publishes pk and keeps sk secret to itself.
- $\mathsf{Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m \in \mathbb{Z}_{2^{r-2}}, \mathsf{pk}) \to \mathsf{ct}$. On input the public parameter $\mathsf{pp}_{\mathsf{bPRE}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E \in \mathscr{E}_p(\mathscr{O})})$, public key $\mathsf{pk} = E_A$ and a message $m \in \mathbb{Z}_{2^{r-2}}$, the sender proceeds to compute a ciphertext ct as follows.
  - (i) The sender embeds the message m in $\mathbb{Z}_{2^r}^\times$ by setting $m' = 2m + 1$.
  - (ii) It samples $(b_1, \ldots, b_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$ and computes the ideal class $[\mathfrak{b}] = \prod_{i=1}^n [\mathfrak{l}_i^{b_i}] \in \mathsf{cl}(\mathscr{O})$.
  - (iii) It computes the class group actions $E_3 = [\mathfrak{b}]E_0, E_4 = [\mathfrak{b}]E_A$, finds a point P_{E_3} of order 2^r using the procedure $\mathsf{FindPoint}(p, r, \ell_1, \ldots, \ell_n, \mathsf{MC}(E_3))$ presented in Algorithm 2 and evaluates $P_3 = [m']P_{E_3}$ where $\mathsf{MC}(E_3)$ is the Montgomery coefficient of the supersingular elliptic curve E_3.

(iv) Finally, it sends the ciphertext $\mathsf{ct} = (E_4, x' = f_{E_3}(x(P_3))$ to the receiver through a public channel.

– $\mathsf{Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{ct} = (E_4, x'), \mathsf{sk} = (a_1, \ldots, a_n)) \to (m \vee \perp)$. After receiving the ciphertext $\mathsf{ct} = (E_4, x')$, the receiver runs this algorithm using its secret key $\mathsf{sk} = (a_1, \ldots, a_n)$ and the public parameter

$$\mathsf{pp}_{\mathsf{bPRE}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E \in \mathscr{E}_p(\mathscr{O})})$$

and returns either a plaintext $m \in \mathbb{Z}_{2^{r-2}}$ or an error symbol $\perp$ by executing the following steps.

(i) The receiver checks wheather E_4 is supersingular or not. It aborts and returns $\perp$ if E_4 is not supersingular.

(ii) Otherwise, it computes the ideal class $[\mathfrak{a}^{-1}] = \prod_{i=1}^{n} [\mathfrak{l}_i^{-a_i}]$ and the class group action $E_3 = [\mathfrak{a}^{-1}]E_4$. It finds a point $P_{E_3} \in E_3$ of order 2^r by invoking the procedure $\mathsf{FindPoint}(p, r, \ell_1, \ldots, \ell_n, \mathsf{MC}(E_3))$ described in Algorithm 2 where $\mathsf{MC}(E_3)$ is the Montgomery coefficient of the supersingular elliptic curve E_3.

(iii) It computes $g_{E_3}(x')$ by extracting x' from ct and returns $\perp$ if $g_{E_3}(x')$ is not the x-coordinate of a 2^r-torsion point on E_3.

(iv) Otherwise, it finds $M \in \mathbb{Z}_{2^r}^{\times}$ by executing the procedure $\mathsf{Pohlig\text{-}Hellman}(E_3, x(P_{E_3}), x(P_3) = g_{E_3}(x'))$ presented in Algorithm 1.

(v) If $2^{r-1} < M$ then it sets $M' = 2^r - M$, else it sets $M' = M$.

(vi) Finally, it returns $m = \frac{M'-1}{2}$.

– $\mathsf{ReKeyGen}\left(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{sk}_i = \left(a_1^{(i)}, \ldots, a_n^{(i)}\right), \mathsf{sk}_j = \left(a_1^{(j)}, \ldots, a_n^{(j)}\right)\right) \to \mathsf{rk}_{i \leftrightarrow j}$. This is a three-party protocol run among a semi-trusted proxy server, a delegator with its secret key $\mathsf{sk}_i = \left(a_1^{(i)}, \ldots, a_n^{(i)}\right)$ and a delegatee with its secret key $\mathsf{sk}_j = \left(a_1^{(j)}, \ldots, a_n^{(j)}\right)$ to produce a re-encryption key $\mathsf{rk}_{i \leftrightarrow j}$ as follows.

(i) The delegator samples $\mathbf{r} = (r_1, \ldots, r_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$ and computes $\mathbf{r} - \mathsf{sk}_i = \left(r_1 - a_1^{(i)}, \ldots, r_n - a_n^{(i)}\right)$. The delegator sends through a secure channel $\mathbf{r}$ to the semi-trusted proxy server and $\mathbf{r} - \mathsf{sk}_i$ to the delegatee.

(ii) Upon receiving $\mathbf{r} - \mathsf{sk}_i$ from delegator, the delegatee computes

$$\mathbf{r} + \mathsf{sk}_j - \mathsf{sk}_i = (\mathbf{r} - \mathsf{sk}_i) + \mathsf{sk}_j = \left(r_1 + a_1^{(j)} - a_1^{(i)}, \ldots, r_n + a_n^{(j)} - a_n^{(i)}\right)$$

and sends through a secure channel $\mathbf{r} + \mathsf{sk}_j - \mathsf{sk}_i$ to the semi-trusted proxy server.

(iii) After receiving $\mathbf{r}$ from delegator and $\mathbf{r} + \mathsf{sk}_j - \mathsf{sk}_i$ from the delegate, the semi-trusted proxy server calculates

$$\mathsf{sk}_j - \mathsf{sk}_i = (\mathbf{r} + \mathsf{sk}_j - \mathsf{sk}_i) - \mathbf{r} = \left(a_1^{(j)} - a_1^{(i)}, \ldots, a_n^{(j)} - a_n^{(i)}\right)$$

and computes the ideal class $\left[\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}\right] = \prod_{k=1}^{n} \left[\mathfrak{l}_k^{\left(a_k^{(j)} - a_k^{(i)}\right)}\right] \in \mathsf{cl}(\mathscr{O})$ by extracting $\ell_1, \ell_2, \ldots, \ell_n$ from

$$\mathsf{pp}_{\mathsf{bPRE}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E \in \mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E \in \mathscr{E}_p(\mathscr{O})})$$

(iv) Finally, the semi-trusted proxy server sets the proxy re-encryption key $\mathsf{rk}_{i\leftrightarrow j} = \left[\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}\right]$.

– $\mathsf{ReEnc}\left(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{rk}_{i\leftrightarrow j} = \left[\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}\right], \mathsf{ct}_i = \left(E_4^{(i)}, x'\right)\right) \to \mathsf{ct}_j = \left(E_4^{(j)}, x'\right)$. Upon receiving a ciphertext ct_i and a re-encryption key $\mathsf{rk}_{i\leftrightarrow j}$, the semi-trusted proxy server returns a ciphertext ct_j by computing the class group action $E_4^{(j)}\mathsf{rk}_{i\leftrightarrow j}E_4^{(i)} = \left[\mathfrak{a}^{(j)}\left(\mathfrak{a}^{(i)}\right)^{-1}\right]E_4^{(i)} = \left[\mathfrak{a}^{(j)}\right]\left[\left(\mathfrak{a}^{(i)}\right)^{-1}\right]E_4^{(i)}$ and returns $\mathsf{ct}_j = (E_4^{(j)}, x')$. We refer ct_i as delegator's ciphertext under the delegator's public key pk_i and ct_j as delegatee's ciphertext under the delegatee's public key pk_j.

Remark 3. Note that the private key in our bidirectional proxy re-encryption bPRE is $(a_1, \ldots, a_n) \xleftarrow{\$} \{-s, \ldots, s\}^n$ in contrast the private key in SimS is the ideal class $[\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}]$. The ciphertext of SimS is of the form $([\mathfrak{b}]E_0, f_{[\mathfrak{b}][\mathfrak{a}]E_0}(x(P_4)))$ where $P_4 = [2m+1]P_{[\mathfrak{a}][\mathfrak{b}]E_0}$, $m \in \mathbb{Z}_{2^{r-2}}$ is a message and $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$. On contrast, the ciphertext in bPRE as $([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x(P_3)))$ with $P_3 = [2m+1]P_{[\mathfrak{b}]E_0}$ for a message $m \in \mathbb{Z}_{2^{r-2}}$ and $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$. This enables a semi-trusted proxy server to re-encrypt a ciphertext using a re-encryption key by invoking the re-encryption algorithm ReEnc of bPRE. More specifically, the semi-trusted proxy server re-encrypts a ciphertext $\mathsf{ct}_i = (E_4^{(i)} = [\mathfrak{b}][\mathfrak{a}^{(i)}]E_0$, $x' = f_{[\mathfrak{b}]E_0}(x(P_3)))$ using a re-encryption key $\mathsf{rk}_{i\leftrightarrow j} = [\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}]$ by computing the class group action $E_4^{(j)} = [\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}][\mathfrak{b}][\mathfrak{a}^{(i)}]E_0 = [\mathfrak{b}][\mathfrak{a}^{(j)}]E_0$ and return the new ciphertext $\mathsf{ct}_j = (E_4^{(i)} = ([\mathfrak{b}][\mathfrak{a}^{(j)}]E_0, x' = f_{[\mathfrak{b}]E_0}(x(P_3)))$ with its second component x' the same as that of ct_i.

Correctness of bPRE Note that the public parameter $\mathsf{pp}_{\mathsf{bPRE}}$ of bPRE is $\mathsf{pp}_{\mathsf{bPRE}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E\in\mathscr{E}_p(\mathscr{O})}) \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$, private and public key pair is $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$ where $\mathsf{pk} = E_A = [\mathfrak{a}]E_0, \mathsf{sk} = (a_1, \ldots, a_n)$ with $[\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}]$ and the ciphertext is $\mathsf{ct} \leftarrow \mathsf{bPRE.Enc}$ $(\mathsf{pp}_{\mathsf{bPRE}}, m \in \mathbb{Z}_{2^{r-2}}, \mathsf{pk})$ where $\mathsf{ct} = (E_4 = [\mathfrak{b}][\mathfrak{a}]E_0, x' = f_{E_3}(x(P_3)), E_3 = [\mathfrak{b}]E_0, P_3 = [2m+1]P_{E_3}$ with $P_{E_3} \in E_3$ is a point of order 2^r. Since f_{E_3} is bijective by the property $(\mathscr{P}1)$ as per Definition 16, the receiver with secret key $\mathsf{sk} = (a_1, \ldots, a_n)$ can compute its inverse $g_{E_3} = f_{E_3}^{-1}$. More concretely, the receiver computes the class group action $E_3 = [\mathfrak{a}^{-1}]E_4$ and calculates $x(P_3) = g_{E_3}(x')$ by extracting x' from ct. The commutativity of the class group $\mathsf{cl}(\mathscr{O})$ implies the curves $[\mathfrak{b}]E_0$ (in the ciphertext) and $[\mathfrak{a}^{-1}][\mathfrak{b}][\mathfrak{a}]E_0$ (computed during the decryption) are equal to E_3. Consequently, the sender and the receiver both get the same point P_{E_3} of order 2^r by the procedure $\mathsf{FindPoint}(p, r, \ell_1, \ldots, \ell_n, \mathsf{MC}(E_3))$ provided in Algorithm 2. As both the points P_{E_3} and $P_3 = [2m+1]P_{E_3}$ have order 2^r, the Pohlig-Hellman algorithm described in Algorithm 1 can be employed on their x-coordinates $x(P_3)$ and $x(P_{E_3})$ to recover $M = \pm(2m+1) \bmod 2^r$. Since $m \in \mathbb{Z}_{2^{r-2}}$, it holds that $2m+1 < 2^{r-1}$. The receiver changes M to $2^r - M$ if $2^{r-1} < M$ and receives plaintext $(M-1)/2 = m$.

Hence it holds that for all λ, all $m \in \mathbb{Z}_{2^{r-2}}$, all $\mathsf{pp}_{\mathsf{bPRE}} \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$ and all $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$,

$$\mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m, \mathsf{pk}), \mathsf{sk}) \rightarrow m$$

Our bidirectional proxy re-encryption scheme bPRE enables a semi-trusted proxy server to convert a ciphertext $\mathsf{ct}_i = \left(E_4^{(i)}, x'\right)$ under the delegator's public key $\mathsf{pk}_i = E_A^{(i)} = [\mathfrak{a}^{(i)}]E_0$ into a ciphertext $\mathsf{ct}_j = \left(E_4^{(j)}, x'\right)$ under the delegatee's public key $\mathsf{pk}_j = E_A^{(j)} = [\mathfrak{a}^{(j)}]E_0$ without disclosing the underlying secret message m to the semi-trusted proxy server by the algorithm $\mathsf{ct}_j = \left(E_4^{(j)}, x'\right) \leftarrow \mathsf{bPRE.ReEnc}\left(\mathsf{rk}_{i\leftrightarrow j} = \left[\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}\right], \mathsf{ct}_i = \left(E_4^{(i)}, x'\right)\right)$ where $\mathsf{rk}_{i\leftrightarrow j} = \left[\mathfrak{a}^{(j)}(\mathfrak{a}^{(i)})^{-1}\right] \leftarrow \mathsf{bPRE.ReKeyGen}\left(\mathsf{sk}_i = \left(a_1^{(i)}, \ldots, a_n^{(i)}\right), \mathsf{sk}_j = \left(a_1^{(j)}, \ldots, a_n^{(j)}\right)\right)$ is the re-encryption key generated by the semi-trusted proxy server. Observe that the delegatee still can compute $E_3 = [\mathfrak{b}]E_0$ as follows by computing the class group action $\left[\left(\mathfrak{a}^{(j)}\right)^{-1}\right] E_4^{(j)}$ using its own secret key $\mathsf{sk}_j = \left(a_1^{(j)}, \ldots, a_n^{(j)}\right)$ and extracting $E_4^{(j)}$ from $\mathsf{ct}_j = \left(E_4^{(j)}, x'\right)$. By the commutativity of the class group

$$\left[\left(\mathfrak{a}^{(j)}\right)^{-1}\right] E_4^{(j)} = \left[\left(\mathfrak{a}^{(j)}\right)^{-1}\right] \left[\mathfrak{a}^{(j)} \left(\mathfrak{a}^{(i)}\right)^{-1}\right] E_4^{(i)} = [\mathfrak{b}]E_0\mathsf{cl}(\mathscr{O})) = E_3$$

It follows that for all λ, for all $m \in \mathbb{Z}_{2^{r-2}}$, for all $\mathsf{pp}_{\mathsf{bPRE}} \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$, for all $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$, for all $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$ and $(\mathsf{sk}_j, \mathsf{pk}_j) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp})$,
$\mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{bPRE.ReEnc}(\mathsf{bPRE.ReKeyGen}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{sk}_i, \mathsf{sk}_j), \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m, \mathsf{pk}_i)), \mathsf{sk}_j) \rightarrow m.$
Hence, the following correctness theorem holds for our bidirectional proxy re-encryption scheme bPRE.

Theorem 11. *Our bidirectional proxy re-encryption scheme* $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$ *associated with message space* $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ *is correct.*

Also, our bidirectional proxy re-encryption bPRE satisfies the multi-hop property which means the semi-trusted proxy server can covert the ciphertexts from Alice to Bob and then to Charlie and so on. This directly follows from the correctness of bPRE and the commutativity of the class group $\mathsf{cl}(\mathscr{O})$ i.e. if we apply the correctness of bPRE, it can be shown for any sequence of secret key and private key pairs $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$ with $1 \leq i \leq N$, all re-encryption keys $\mathsf{rk}_{j\leftrightarrow j+1} \leftarrow \mathsf{bPRE.ReKeyGen}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{sk}_j, \mathsf{sk}_{j+1})$ with $j < N$, all messages $m \in \mathbb{Z}_{2^{r-2}}$ it holds that
$\mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{bPRE.ReEnc}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{rk}_{N-1\leftrightarrow N}, \ldots, \mathsf{bPRE.ReEnc}(\mathsf{pp}_{\mathsf{bPRE}}, \mathsf{rk}_{1\leftrightarrow 2}, \mathsf{ct}_1)), \mathsf{sk}_N) \rightarrow m$
where $\mathsf{ct}_1 \leftarrow \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{bPRE}}, m, \mathsf{pk}_1)$ and $\mathsf{pp}_{\mathsf{bPRE}} \leftarrow \mathsf{bPRE.Setup}(1^\lambda)$. Hence, we have the following theorem.

Theorem 12. *Our bidirectional proxy re-encryption scheme* bPRE = (Setup, KeyGen, Enc, Dec, $\mathsf{ReKeyGen}$, ReEnc) *associated with message space* $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ *is multi-hop.*

Security analysis of bPRE

Theorem 13. *If Assumption 2 (CSSIDDH) holds and the family of randomizing functions* $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ *indexed by supersingular elliptic curves satisfies the property* ($\mathscr{P}2$) *as per Definition 16, then our bidirectional proxy re-encryption* bPRE = (Setup, KeyGen, Enc, Dec,$\mathsf{ReKeyGen}$, ReEnc) *associated with message space* $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ *is IND-CPA secure as per Definition 19.*

Proof. Assume that our bidirectional proxy re-encryption scheme bPRE = (Setup, KeyGen, Enc, Dec,$\mathsf{ReKeyGen}$, ReEnc) is not IND-CPA secure. Then there exists a PPT adversary $\mathscr{A}$ such that

$$\mathsf{Adv}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda) = \left|\Pr[\mathsf{Exp}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda) = 1] - \frac{1}{2}\right| > \delta,$$

where δ is a non-negligible function of the security parameter λ. We will construct a CSSIDDH distinguisher $\mathscr{B}$ using the adversary $\mathscr{A}$ as a simulator that will break the Assumption 2.

Let $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ be a tuple given by a CSSIDDH challenger $\mathscr{D}$ to $\mathscr{B}$ where $[\mathfrak{c}], [\mathfrak{d}] \in \mathsf{cl}(\mathscr{O})$, $E_0 : y^2 = x^3 + x$ is the supersingular elliptic curve over $\mathbb{F}_p$ and $E \in \mathscr{E}_p(\mathscr{O})$. Our aim is to show how the distinguisher $\mathscr{B}$ exploits the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ to decide if E is $[\mathfrak{c}][\mathfrak{d}]E_0$ (real instance) or $[\mathfrak{r}]E_0$ (random instance) for some $[\mathfrak{r}] \in \mathsf{cl}(\mathscr{O})$ using $\mathscr{A}$ as a subroutine. We will use the distinguished $\mathscr{B}$ as a challenger in the experiment $\mathsf{Exp}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}$ with the adversary $\mathscr{A}$ as follows:

Phase 0:

- **Setup:** The distinguisher $\mathscr{B}$ extracts E_0 from the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ and sets

$$\mathsf{pp}_{\mathsf{bPRE}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E\in\mathscr{E}_p(\mathscr{O})})$$

  and sends $\mathsf{pp}_{\mathsf{bPRE}}$ to $\mathscr{A}$. Here $p = 2^r\ell_1\ell_2\cdots\ell_n - 1$ is a prime number with $\lambda+2 \le r \le \frac{1}{2}\log p$, $\ell_1, \ell_2, \ldots, \ell_n$ are n small distinct odd prime numbers, s is an integer satisfying $(2s+1)^n \ge |\mathsf{cl}(\mathscr{O})|$, $\{f_E\}_{E\in\mathsf{cl}(\mathscr{O})}$ and $\{g_E\}_{E\in\mathsf{cl}(\mathscr{O})}$ are two families of randomizing functions indexed by supersingular elliptic curves satisfying $g_E = f_E^{-1}$. Note that $\mathsf{pp}_{\mathsf{bPRE}}$ is identically distributed as in the real protocol by the algorithm $\mathsf{bPRE.Setup}(1^\lambda)$.

Phase 1:

- **Uncorrupted key generation** $\mathscr{O}_{Uncorrupt}$: Upon this query on an user index i from $\mathscr{A}$ with $i \notin \Gamma_U \cup \Gamma_C$, the distinguished $\mathscr{B}$ generates a key pair ($\mathsf{pk} = E_A, \mathsf{sk} = (a_1, \ldots, a_n)$) $\leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{bPRE}})$ with $(a_1, \ldots, a_n) \in \{-s, \ldots, s\}^n, E_A = [\mathfrak{a}]E_0 \in \mathscr{E}_p(\mathscr{O}), [\mathfrak{a}] = \prod_{i=1}^n[\mathfrak{l}_i^{a_i}]$

$\in \mathsf{cl}(\mathscr{O})$, $\mathfrak{l}_i = \langle \ell_i, \pi - 1\rangle$ for $i = 1, 2 \ldots, n$ and π is the Frobenius map. Then the distinguisher $\mathscr{B}$ sends E_A to $\mathscr{A}$ and adds i in Γ_U. Here Γ_U (initially empty) is the list of indices of uncorrupted users and Γ_C (initially empty) is the list of indices of corrupted users maintained by $\mathscr{B}$. Generation of $(\mathsf{sk}, \mathsf{pk})$ pair is exactly same as that in the real protocol and consequently, $(\mathsf{sk}, \mathsf{pk})$ pairs have identical distribution as in the real protocol.

- **Corrupted key generation** $\mathscr{O}_{Corrupt}$: Upon this query on an user index i from $\mathscr{A}$ with $i \notin \Gamma_U \cup \Gamma_C$, the distinguished $\mathscr{B}$ generates a key pair $(\mathsf{pk} = E_A, \mathsf{sk} = (a_1, \ldots, a_n)) \leftarrow \mathsf{bPRE.KeyGen}(\mathsf{pp_{bPRE}})$ with $(a_1, \ldots, a_n) \in \{-s, \ldots, s\}^n, E_A = [\mathfrak{a}]E_0 \in \mathscr{E}_p(\mathscr{O}), [\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}] \in \mathsf{cl}(\mathscr{O})$, $\mathfrak{l}_i = \langle \ell_i, \pi - 1\rangle$ for $i = 1, 2 \ldots, n$ and π is the Frobenius map. It adds i in Γ_C and sends $(\mathsf{sk}, \mathsf{pk})$ to $\mathscr{A}$. Generation of $(\mathsf{sk}, \mathsf{pk})$ pair is exactly same as that in the real protocol and consequently, $(\mathsf{sk}, \mathsf{pk})$ pairs have identical distribution as in the real protocol.

Phase 2:

- **Re-encryption key generation** $\mathscr{O}_{Rkgen}$: On query (i, j) from the adversary $\mathscr{A}$, the distinguisher $\mathscr{B}$ checks both $i, j \in \Gamma_U$ or both $i, j \in \Gamma_C$. If not $\mathscr{B}$ returns $\perp$. Otherwise, the distinguisher $\mathscr{B}$ computes the re-encryption key
$\mathsf{rk}_{i \leftrightarrow j} = \mathsf{sk}_j - \mathsf{sk}_i = \left(a_1^{(j)} - a_1^{(i)}, \cdots, a_n^{(j)} - a_n^{(i)}\right) \leftarrow \mathsf{bPRE.ReKeyGen}(\mathsf{pp_{bPRE}}, \mathsf{sk}_i, \mathsf{sk}_j)$
and sends $\mathsf{rk}_{i \leftrightarrow j}$ to $\mathscr{A}$ where $\mathsf{sk}_i = \left(a_1^{(i)}, \cdots, a_n^{(i)}\right)$ and $\mathsf{sk}_j = \left(a_1^{(j)}, \cdots, a_n^{(j)}\right)$ are the secret keys corresponding to $\mathsf{pk}_i = E_A^{(i)} = \prod_{k=1}^{n}\left[\mathfrak{l}_k^{a_k^{(i)}}\right]$ and $\mathsf{pk}_j = E_A^{(j)} = \prod_{k=1}^{n}\left[\mathfrak{l}_k^{a_k^{(j)}}\right]$ respectively. The distribution of $\mathsf{rk}_{i \leftrightarrow j}$ is aexacly same as in real protocol.
- **Challenge Phase:** The distinguisher $\mathscr{B}$ sets the challenge public key $\mathsf{pk}_{i^*} = [\mathfrak{c}]E_0$ by extracting $[\mathfrak{c}]E_0$ from the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$, adds i^* in Γ_U and sends pk_{i^*} to $\mathscr{A}$. The adversary $\mathscr{A}$ sends a message pair $(m_0, m_1) \in \mathscr{M}^2$ to $\mathscr{B}$. The distinguisher $\mathscr{B}$ chooses $\mu \xleftarrow{\$} \{0, 1\}$, sets $E' = [\mathfrak{d}]E_0$ by extracting $[\mathfrak{d}]E_0$ from CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$, computes the point $P_{E'} \in E'$ of order 2^r by invoking the procedure $P_{E'} \leftarrow \mathsf{FindPoint}(p, r, \ell_1, \ell_2, \ldots, \ell_n, \mathsf{MC}(E'))$ given in Algorithm 2 by extracting $p, \ell_1, \ell_2, \ldots, \ell_n, r$ from $\mathsf{pp_{bPRE}}$ and sets the challenge ciphertext $\mathsf{ct}^* = (E, f_{E'}(x([2m_\mu + 1]P_{E'})))$ by extracting E from CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$. The adversary $\mathscr{A}'$ sends the challenge ciphertext ct^* to $\mathscr{A}$.

Phase 3:

- **Decision phase:** The adversary $\mathscr{A}$ guess a bit $\mu^* \in \{0, 1\}$ for $\mu \in \{0, 1\}$ and sends μ^* to $\mathscr{B}$. If $\mu = \mu^*$, the distinguisher $\mathscr{B}$ sends $\beta = 1$ to the CSSIDDH challenger $\mathscr{D}$. Otherwise, $\mathscr{B}$ sends $\beta = 0$ to $\mathscr{D}$.

Note that We have the following two cases:

– **Case 1.** If the instance given to the distinguisher $\mathscr{B}$ by the CSSIDDH challenger $\mathscr{D}$ is real instance of the form $(E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E = [\mathfrak{d}][\mathfrak{c}]E_0)$ where $[\mathfrak{c}], [\mathfrak{d}] \in \mathsf{cl}(\mathscr{O})$, then the simulation of $\mathsf{pp}_{\mathsf{bPRE}}$, the challenge public key pk_{i^*} and the challenge ciphertext $\mathsf{ct}^* = (E, f_{[\mathfrak{b}]E_0}(x([2m_\mu + 1]P_{[b]E_0})))$ exactly identical as those in the original protocol and $\mu = \mu^*$ with high probability.

– **Case 2.** If the instance given to $\mathscr{B}$ is random instance of the form $(E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E = [\mathfrak{r}]E_0)$ where $[\mathfrak{c}], [\mathfrak{d}], [\mathfrak{r}] \in \mathsf{cl}(\mathscr{O})$ then the simulation of $\mathsf{pp}_{\mathsf{bPRE}}$ and the challenge public key pk_{i^*} are identical to the real protocol but the ciphertext ct^* is of the form $\mathsf{ct}^* = ([\mathfrak{r}]E_0, f_{[[\mathfrak{b}]E_0}(x([2m_\mu + 1]P_{[\mathfrak{b}]E_0})))$ and $\mu \neq \mu^*$ with overwhelming probability as $\mathscr{A}$ guess μ with probability $\frac{1}{2}$.

The above two cases imply that if the advantage $\mathsf{Adv}^{\mathsf{CPA}}_{\mathscr{A},\mathsf{bPRE}}(1^\lambda)$ of the adversary $\mathscr{A}$ is non-negligible, then the advantage of the CSSIDDH adversary $\mathscr{B}$ is also non-negligible. i.e.

$$\left| \Pr\left[\mu = \mu^* \middle| \begin{array}{l} [\mathfrak{c}], [\mathfrak{d}], [\mathfrak{r}] \leftarrow \mathsf{cl}(\mathscr{O}), \mu \xleftarrow{\$} \{0,1\}, \\ F_0 = [\mathfrak{d}][\mathfrak{c}]E_0, F_1 = [\mathfrak{r}]E_0, \\ \mu^* \leftarrow \mathscr{B}(E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, F_b) \end{array} \right] - \frac{1}{2} \right| > \delta$$

This proved that our bidirectional proxy re-encyption $\mathsf{bPRE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{ReKeyGen}, \mathsf{ReEnc})$ is IND-CPA secure.

□

3.2 MSimS: An IND-CCA secure Public Key Encryption

Note that the algorithms $\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}$ of our bidirectional proxy re-encryption scheme bPRE constitute a public key encryption scheme which is a variant of the encryption scheme SimS presented in Algorithm 3, 4, 5, 6. We refer this modified SimS as $\mathsf{MSimS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ with message space $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ where

– $\mathsf{MSimS.Setup}(1^\lambda) = \mathsf{bPRE.Setup}(1^\lambda) \rightarrow \mathsf{pp}_{\mathsf{bPRE}} = \mathsf{pp}_{\mathsf{MSimS}}$
– $\mathsf{MSimS.KeyGen}(\mathsf{pp}_{\mathsf{MSimS}}) = \mathsf{bPRE.KeyGen}(\mathsf{pp}_{\mathsf{MSimS}}) \rightarrow (\mathsf{sk}, \mathsf{pk})$
– $\mathsf{MSimS.Enc}(\mathsf{pp}_{\mathsf{MSimS}}, m \in \mathbb{Z}_{2^{r-2}}, \mathsf{pk}) = \mathsf{bPRE.Enc}(\mathsf{pp}_{\mathsf{MSimS}}, m \in \mathscr{M}, \mathsf{pk}) \rightarrow \mathsf{ct}$
– $\mathsf{MSimS.Dec}(\mathsf{pp}_{\mathsf{MSimS}}, \mathsf{ct}, \mathsf{sk}) = \mathsf{bPRE.Dec}(\mathsf{pp}_{\mathsf{MSimS}}, \mathsf{ct}, \mathsf{sk}) \rightarrow (m \vee \perp)$

Correctness of MSimS The following correctness theorem holds for the public key encryption scheme MSimS which follows from the correctness of our bPRE (Theorem 12).

Theorem 14. *The public key encryption scheme* $\mathsf{MSimS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *associated with message space* $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ *is correct if the family of randomizing functions* $\{f_{E_3}\}_{E \in \mathscr{E}_p(\mathscr{O})}$ *satisfies the property* $(\mathscr{P}1)$ *as per Definition 16.*

Security analysis of MSimS

Theorem 15. *If Assumption 2 (CSSIDDH) holds and the family of randomizing functions $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ indexed by supersingular elliptic curves satisfies the property ($\mathscr{P}2$) as per Definition 16, then* $\mathsf{MSimS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *associated with the message space $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ is IND-CPA secure as per Definition 2.*

Proof. Suppose that MSimS is not IND-CPA secure. Then there exists a PPT adversary $\mathscr{A}$ against MSimS that can distinguish whether a given challenge ciphertext $\mathsf{ct}^* = (E_4 = [\mathfrak{b}][\mathfrak{a}]E_0, x' = f_{E_3}(x([2m_\mu + 1]P_{E_3}))$ is an encryption of plaintext $m_0 \in \mathscr{M}$ or $m_1 \in \mathscr{M}$ under a public key $\mathsf{pk} = [\mathfrak{a}]E_0$ with a non-negligible advantage δ where $\mathscr{A}$ is given access to the public parameter $\mathsf{pp}_{\mathsf{MSimS}}$, message space$\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ and public key pk by a MSimS challenger $\mathscr{C}$. Here $\mathsf{pp}_{\mathsf{MSimS}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E\in\mathscr{E}_p(\mathscr{O})}) \leftarrow \mathsf{MSimS.Setup}(1^\lambda)$, $(\mathsf{sk} = (a_1, \ldots, a_n), \mathsf{pk} = [\mathfrak{a}]E_0) \leftarrow \mathsf{MSimS.KeyGen}(\mathsf{pp}_{\mathsf{MSimS}})$ are generated by a MSimS challenger $\mathscr{C}$, (m_0, m_1) is the challenge plaintext pair submitted by $\mathscr{A}$ to $\mathscr{C}$, $p = 2^r\ell_1\cdots\ell_n - 1$ is a prime number, $\ell_1, \ldots, \ell_n$ are small odd primes, r is an integer such that $\lambda + 2 \le r \le \frac{1}{2}\log p$, s is an integer such that $(2s+1)^n \ge |\mathsf{cl}(\mathscr{O})|$, $E_0 : y^2 = x^3 + x$ is the supersingular elliptic curve over $\mathbb{F}_p$, $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ is a family of randomizing functions indexed by supersingular elliptic curves with its inverse $g_E = f_E^{-1}$, $[\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$, $E_3 = [\mathfrak{b}]E_0$ and $P_{E_3} \in E_3$ is a point of 2^r and $[\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}]$ where $\mathfrak{l}_i = \langle \ell_i, \pi - 1\rangle$ for $i = 1, 2\ldots, n$ and π is the Frobenius map. We will show below how to utilise $\mathscr{A}$ to construct a CSSIDDH distinguisher $\mathscr{A}'$ that breaks Assumption 2.

Let $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ be a tuple given by a CSSIDDH challenger $\mathscr{C}'$ to $\mathscr{A}'$ where $[\mathfrak{c}], [\mathfrak{d}] \in \mathsf{cl}(\mathscr{O})$, $E_0 : y^2 = x^3 + x$ is the supersingular elliptic curve over $\mathbb{F}_p$ and $E \in \mathscr{E}_p(\mathscr{O})$. Our aim is to show how the adversary $\mathscr{A}'$ exploits the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ to decide if E is $[\mathfrak{c}][\mathfrak{d}]E_0$ (real instance) or $[\mathfrak{r}]E_0$ (random instance) for some $[\mathfrak{r}] \in \mathsf{cl}(\mathscr{O})$ using $\mathscr{A}$ as a subroutine. The distinguisher $\mathscr{A}'$ plays the role of a challenger for $\mathscr{A}$ and does the following to simulate $\mathscr{A}$.

- It sets the public parameter $\mathsf{pp}_{\mathsf{MSimS}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E\in\mathscr{E}_p(\mathscr{O})})$ and sends it to $\mathscr{A}$ where $p = 2^r\ell_1\ell_2\cdots\ell_n - 1$ is a prime number with $\lambda + 2 \le r \le \frac{1}{2}\log p$, $\ell_1, \ell_2, \ldots, \ell_n$ are n small distinct odd prime numbers, s is an integer satisfying $(2s+1)^n \ge |\mathsf{cl}(\mathscr{O})|$, $\{f_E\}_{E\in\mathsf{cl}(\mathscr{O})}$ and $\{g_E\}_{E\in\mathsf{cl}(\mathscr{O})}$ are two families of randomizing functions indexed by supersingular elliptic curves satisfying $g_E = f_E^{-1}$ and E_0 is extracted from the CSSIDDH instance I.
- It sets the public key $\mathsf{pk} = [\mathfrak{c}]E_0$ by extracting $[\mathfrak{c}]E_0$ from the CSSIDDH instance $I = (E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$.
- On receiving a challenge message pair (m_0, m_1) from $\mathscr{A}$, it chooses $\mu \xleftarrow{\$} \{0, 1\}$, sets $E' = [\mathfrak{d}]E_0$, computes the point $P_{E'} \in E'$ of order 2^r by using the procedure $\mathsf{FindPoint}(p, r, \ell_1, \ell_2, \ldots, \ell_n, \mathsf{MC}(E'))$ given in Algorithm 2 and sets the challenge ciphertext $\mathsf{ct}^* = (E, f_{[E'}(x([2m_\mu + 1]P_{E'})))$. The distinguisher $\mathscr{A}'$ sends the challenge ciphertext ct^* to $\mathscr{A}$ and receives $\mu^* \in \{0, 1\}$ from $\mathscr{A}$ in response.

- Upon receiving μ^* from $\mathscr{A}$, the distinguisher $\mathscr{A}'$ submits 1 to the CSSIDDH challenger $\mathscr{C}'$ if $\mu = \mu^*$. Otherwise, it submits 0 to $\mathscr{C}'$.

We have the following two cases:

- **Case 1.** If the instance given to the distinguisher $\mathscr{A}'$ by the CSSIDDH challenger $\mathscr{C}$ is real instance of the form $(E_0, [\mathfrak{d}]E_0, [\mathfrak{c}]E_0, E = [\mathfrak{c}][\mathfrak{d}]E_0)$ then the simulations of $\mathsf{pp}_{\mathsf{MSimS}}$, pk, ct^* are exactly identical as those in the original protocol and $\mu = \mu^*$ with high probability.
- **Case 2.** If the instance given to $\mathscr{A}'$ is random instance of the form $(E_0, [\mathfrak{c}]E_0,$ $[\mathfrak{d}]E_0, E = [\mathfrak{r}]E_0)$ then the simulation of $\mathsf{pp}_{\mathsf{MSimS}}$ and pk are identical to that of the real protocol. However, the simulated ciphertext ct^* is of the form $\mathsf{ct} = ([\mathfrak{r}]E_0, f_{[[\mathfrak{d}]E_0}(x([2m_\mu + 1]P_{[\mathfrak{d}]E_0})))$ and $\mu \neq \mu^*$ with overwhelming probability as $\mathscr{A}$ guess μ with probability $\frac{1}{2}$.

Thus, if the IND-CPA adversary $\mathscr{A}$ against MSimS has a non-negligible advantage δ in guessing the bit μ correctly used in creating the challenge ciphertext ct^* by the IND-CPA challenger $\mathscr{A}'$ then $\mathscr{A}'$ will be able to distinguish whether the instance $(E_0, [\mathfrak{c}]E_0, [\mathfrak{d}]E_0, E)$ given by the CSSIDDH challenger $\mathscr{C}'$ with the same non-negligible advantage δ. This established our encryption scheme MSimS is IND-CPA secure.

□

To prove the IND-CCA security of the public-key encryption scheme SimS, Fouotsa et al. [12] introduced Commutative Supersingular Isogeny Knowledge of Exponent (CSSIKOE) which we described in Assumption 3. To establish the IND-CCA security of our encryption scheme MSimS we describe the CSSIKOE assumption in our notations and rename it as Commutative Supersingular Isogeny Knowledge of Exponent II (CSSIKOE-II). In our construction MSimS, the ciphertext is of the form $\mathsf{ct} = ([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$ where $[\mathfrak{a}], [\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$. Intuitively, this assumption states that the only effective way to compute a valid ciphertext is to first fix the ideal class $[\mathfrak{b}]$ then compute $\mathsf{ct} = ([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x([2m+1]P_{[\mathfrak{b}]E_0})))$.

Assumption 4 (CSSIKOE-II)**.** *Let λ be a security parameter and $p = 2^r\ell_1 \cdots \ell_n - 1$ be a prime such that $\lambda + 2 \leq r \leq \frac{1}{2}\log p$, where $\ell_1, \ldots, \ell_n$ are small odd primes. Let $(a_1, \ldots, a_n)$ and $(b_1, \ldots, b_n)$ are uniformly sampled from $\{-s, \ldots, s\}^n$. Define $[\mathfrak{a}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{a_i}]$, $[\mathfrak{b}] = \prod_{i=1}^{n}[\mathfrak{l}_i^{b_i}] \in \mathsf{cl}(\mathscr{O})$. Let $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ be a family of randomizing functions indexed by supersingular elliptic curves as per Definition 16. The Commutative Supersingular Isogeny Knowledge of Exponent II (CSSIKOE-II) assumption states that for every PPT adversary $\mathscr{A}$, if $\mathscr{A}(E_0, [\mathfrak{a}]E_0,$ $([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x(P)))) \to ([\mathfrak{b}'][\mathfrak{a}]E_0, f_{[\mathfrak{b}']E_0}(x(P'))) \neq ([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x(P)))$ where $P \in [\mathfrak{b}]E_0$ is a point of order 2^r, $[\mathfrak{b}'] \in \mathsf{cl}(\mathscr{O})$ and $P' \in [\mathfrak{b}']E_0$ is a point of order 2^r, then there exists a PPT distinguisher $\mathscr{A}'$ such that $\mathscr{A}'(E_0, [\mathfrak{a}]E_0,$ $([\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x(P)))) \to ([\mathfrak{b}'], [\mathfrak{b}'][\mathfrak{a}]E_0, f_{[\mathfrak{b}']E_0}(x(P')))$.*

Theorem 16. *If Assumption 2 and Assumption 4 hold and the family of randomizing functions $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ indexed by supersingular elliptic curves satisfies the property $(\mathscr{P}2)$ as per Definition 16, then the public-key encryption* *scheme* $\mathsf{MSimS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *associated with the message space* $\mathscr{M} = \mathbb{Z}_{2^{r-2}}$ *is IND-CCA secure as per Definition 3.*

Proof. If Assumption 2 holds and the family of randomizing functions $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ indexed by supersingular elliptic curves satisfies the property $(\mathscr{P}2)$, then MSimS is IND-CPA secure by Theorem 15. Let us now suppose that Assumption 4 holds. We assume there exists an IND-CCA adversary $\mathscr{A}^{\mathscr{O}(\cdot)} = (\mathscr{A}_1, \mathscr{O}(\cdot))$ against MSimS which can successfully guess the bit bit $\mu \in \{0,1\}$ randomly selected by the IND-CCA challenger $\mathscr{C}$ used in forming the challenge ciphertext $\mathsf{ct}^* = (E_4 = [\mathfrak{b}][\mathfrak{a}]E_0, x' = f_{E_3}(x([2m_\mu + 1]P_{E_3})) \leftarrow \mathsf{MSimS.Enc}(\mathsf{pp}_{\mathsf{MSimS}}, m_\mu, \mathsf{pk})$ under a public key $\mathsf{pk} = [\mathfrak{a}]E_0$ with a non-negligible advantage γ. Here $\mathsf{pp}_{\mathsf{MSimS}} = (p, \ell_1, \ell_2, \ldots, \ell_n, r, s, E_0, \{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}, \{g_E\}_{E\in\mathscr{E}_p(\mathscr{O})}) \leftarrow \mathsf{MSimS.Setup}(1^\lambda)$, $(\mathsf{sk} = (a_1, \ldots, a_n), \mathsf{pk} = [\mathfrak{a}]E_0) \leftarrow \mathsf{MSimS.KeyGen}(\mathsf{pp}_{\mathsf{MSimS}})$ are generated by a MSimS challenger $\mathscr{C}$ who gives the access of the public parameter $\mathsf{pp}_{\mathsf{MSimS}}$ and the public key pk to $\mathscr{A}^{\mathscr{O}(\cdot)}$ before setting the challenge ciphertext ct^*. The IND-CCA adversary $\mathscr{A}^{\mathscr{O}(\cdot)}$ has access to decryption oracle $\mathscr{O}(\cdot)$ under the secret key sk, (m_0, m_1) is the challenge plaintext pair provided by $\mathscr{A}^{\mathscr{O}(\cdot)}(\mathsf{pp}_{\mathsf{MSimS}}, \mathscr{M}, \mathsf{pk})$ to the challenger $\mathscr{C}$, $p = 2^r\ell_1 \cdots \ell_n - 1$ is a prime number, $\ell_1, \ldots, \ell_n$ are small odd primes, r is an integer such that $\lambda + 2 \leq r \leq \frac{1}{2}\log p$, s is an integer such that $(2s+1)^n \geq |\mathsf{cl}(\mathscr{O})|$, $E_0 : y^2 = x^3 + x$ is the base supersingular elliptic curve over $\mathbb{F}_p$, $\{f_E\}_{E\in\mathscr{E}_p(\mathscr{O})}$ is a family of randomizing functions indexed by supersingular elliptic curves with its inverse $g_E = f_E^{-1}$, $[\mathfrak{b}] \in \mathsf{cl}(\mathscr{O})$, $E_3 = [\mathfrak{b}]E_0$ and $P_{E_3} \in E_3$ is a point of 2^r . Then we will show that MSimS is not IND-CPA secure. In particular, we will construct a PPT adversary $\mathscr{A}'$ that can determine whether the challenge ciphertext ct^* is an encryption of m_0 or m_1 without using the decryption oracle $\mathscr{O}(\cdot)$.

Let $\mathscr{A}^{\mathscr{O}(\cdot)}$ queries the decryption oracle $\mathscr{O}(\cdot)$ with k many valid ciphertexts $\mathsf{ct}_1 = ([\mathfrak{b}_1][\mathfrak{a}]E_0, f_{[\mathfrak{b}_1]E_0}(x(P_i))), \ldots, \mathsf{ct}_k = ([\mathfrak{b}_k][\mathfrak{a}]E_0, f_{[\mathfrak{b}_k]E_0}(x(P_k))))$ where $P_i \in [\mathfrak{b}_i]E_0$ is a point of order 2^r for $i = 1, 2\ldots, k$. Then by Assumption 4, there exists a PPT adversary $\mathscr{A}_2$ that on input $(E_0, [\mathfrak{a}]E_0, [\mathfrak{b}][\mathfrak{a}]E_0, f_{[\mathfrak{b}]E_0}(x(P))$ can output $([\mathfrak{b}_1], [\mathfrak{b}_1][\mathfrak{a}]E_0 f_{[\mathfrak{b}_1]E_0}(x(P_i))), \ldots, ([\mathfrak{b}_k], [\mathfrak{b}_k][\mathfrak{a}]E_0, f_{[\mathfrak{b}_k]E_0}(x(P_k))))$ Using the knowledge of ideal classes $[\mathfrak{b}_1], \ldots, [\mathfrak{b}_n]$, the adversary $\mathscr{A}_2$ can decrypt the ciphertexts $\mathsf{ct}_1 = ([\mathfrak{b}_1][\mathfrak{a}]E_0, f_{[\mathfrak{b}_1]E_0}(x(P_i))), \cdots, \mathsf{ct}_k = ([\mathfrak{b}_k][\mathfrak{a}]E_0, f_{[\mathfrak{b}_k]E_0}(x(P_k))))$ correctly. Thus, replacing the decryption oracle $\mathscr{O}(\cdot)$ of $\mathscr{A}^{\mathscr{O}(\cdot)}$ by the adversary $\mathscr{A}_2$, we get an adversary $\mathscr{A}' = (\mathscr{A}_1, \mathscr{A}_2)$ which has the ability to determine whether the challenge ciphertext ct^* given by IND-CCA challenger $\mathscr{C}$ is encrypted from a plaintext m_0 or m_1 with non-negligible advantage γ without using the decryption oracle. This contradicts MSimS is IND-CPA secure. This proves that MSimS is IND-CCA secure. □

References

1. Giuseppe Ateniese, Kevin Fu, Matthew Green, and Susan Hohenberger. Improved proxy re-encryption schemes with applications to secure distributed storage. *ACM Transactions on Information and System Security (TISSEC)*, 9(1):1–30, 2006.

2. Matt Blaze, Gerrit Bleumer, and Martin Strauss. Divertible protocols and atomic proxy cryptography. In Kaisa Nyberg, editor, *Advances in Cryptology — EUROCRYPT'98*, pages 127–144, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.
3. Emmanuel Bresson, Dario Catalano, and David Pointcheval. A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 37–54. Springer, 2003.
4. Ran Canetti and Susan Hohenberger. Chosen-ciphertext secure proxy re-encryption. In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 185–194, 2007.
5. Wouter Castryck and Thomas Decru. Csidh on the surface. In *International Conference on Post-Quantum Cryptography*, pages 111–129. Springer, 2020.
6. Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: An efficient post-quantum commutative group action. In Thomas Peyrin and Steven D. Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018*, pages 395–427. Springer International Publishing, 2018.
7. Yun-Peng Chiu, Chin-Laung Lei, and Chun-Ying Huang. Secure multicast using proxy encryption. In *International Conference on Information and Communications Security*, pages 280–290. Springer, 2005.
8. Sherman SM Chow, Jian Weng, Yanjiang Yang, and Robert H Deng. Efficient unidirectional proxy re-encryption. In *Progress in Cryptology–AFRICACRYPT 2010: Third International Conference on Cryptology in Africa, Stellenbosch, South Africa, May 3-6, 2010. Proceedings 3*, pages 316–332. Springer, 2010.
9. Christina Delfs and Steven D Galbraith. Computing isogenies between supersingular elliptic curves over f _p f p. *Designs, Codes and Cryptography*, 78:425–440, 2016.
10. Robert H Deng, Jian Weng, Shengli Liu, and Kefei Chen. Chosen-ciphertext secure proxy re-encryption without pairings. In *Cryptology and Network Security: 7th International Conference, CANS 2008, Hong-Kong, China, December 2-4, 2008. Proceedings 7*, pages 1–17. Springer, 2008.
11. Xiong Fan and Feng Hao Liu. Proxy re-encryption and re-signatures from lattices. In *17th International Conference on Applied Cryptography and Network Security, ACNS 2019*, pages 363–382. Springer Verlag, 2019.
12. Tako Boris Fouotsa and Christophe Petit. Sims: a simplification of sigamal. In *Post-Quantum Cryptography: 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20–22, 2021, Proceedings 12*, pages 277–295. Springer, 2021.
13. Eiichiro Fujisaki and Tatsuaki Okamoto. How to enhance the security of public-key encryption at minimum cost. In *International Workshop on Public Key Cryptography*, pages 53–68. Springer, 1999.
14. Matthew Green and Giuseppe Ateniese. Identity-based proxy re-encryption. In *International Conference on Applied Cryptography and Network Security*, pages 288–306. Springer, 2007.
15. Thomas S Heydt-Benjamin, Hee-Jin Chae, Benessa Defend, and Kevin Fu. Privacy for public transportation. In *International Workshop on Privacy Enhancing Technologies*, pages 1–19. Springer, 2006.
16. Jeffrey Hoffstein, Jill Pipher, and Joseph H Silverman. Ntru: A ring-based public key cryptosystem. In *International algorithmic number theory symposium*, pages 267–288. Springer, 1998.
17. Anca-Andreea Ivan and Yevgeniy Dodis. Proxy cryptography revisited. In *NDSS*, volume 3, pages 1–20. Citeseer, 2003.

18. Jonathan Katz and Yehuda Lindell. *Introduction to modern cryptography: principles and protocols.* Chapman and hall/CRC, 2007.
19. Himanshu Khurana, Adam Slagell, and Rafael Bonilla. Sels: a secure e-mail list service. In *Proceedings of the 2005 ACM symposium on Applied computing*, pages 306–313, 2005.
20. Elena Kirshanova. Proxy re-encryption from lattices. In *Public-Key Cryptography–PKC 2014: 17th International Conference on Practice and Theory in Public-Key Cryptography, Buenos Aires, Argentina, March 26-28, 2014. Proceedings 17*, pages 77–94. Springer, 2014.
21. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 700–718. Springer, 2012.
22. Tomoki Moriya, Hiroshi Onuki, and Tsuyoshi Takagi. Sigamal: a supersingular isogeny-based pke and its application to a prf. In *Advances in Cryptology–ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II 26*, pages 551–580. Springer, 2020.
23. David Nunez, Isaac Agudo, and Javier Lopez. Ntrureencrypt: An efficient proxy re-encryption scheme based on ntru. In *Proceedings of the 10th ACM symposium on information, computer and communications security*, pages 179–189, 2015.
24. Katsuyuki Okeya, Hiroyuki Kurumatani, and Kouichi Sakurai. Elliptic curves with the montgomery-form and their cryptographic applications. In *Public Key Cryptography: Third International Workshop on Practice and Theory in Public Key Cryptosystems, PKC 2000, Melbourne, Victoria, Australia, January 18-20, 2000. Proceedings 3*, pages 238–257. Springer, 2000.
25. Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In *Annual international cryptology conference*, pages 554–571. Springer, 2008.
26. Stephen Pohlig and Martin Hellman. An improved algorithm for computing logarithms over gf (p) and its cryptographic significance (corresp.). *IEEE Transactions on information Theory*, 24(1):106–110, 1978.
27. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
28. Jun Shao and Zhenfu Cao. Cca-secure proxy re-encryption without pairings. In *International Workshop on Public Key Cryptography*, pages 357–376. Springer, 2009.
29. Jun Shao, Rongxing Lu, Xiaodong Lin, and Kaitai Liang. Secure bidirectional proxy re-encryption for cryptographic cloud storage. *Pervasive and Mobile Computing*, 28:113–121, 2016.
30. Carl Siegel. Über die classenzahl quadratischer zahlkörper. *Acta Arithmetica*, 1(1):83–86, 1935.
31. J.H. Silverman. *The Arithmetic of Elliptic Curves.* Graduate Texts in Mathematics. Springer New York, 2009.
32. T. Smith. DVD jon: Buy DRM-less tracks from Apple iTunes. `http://www.theregister.co.uk/2005/03/18/itunespymusique`, January 2005.
33. Ian Stewart and David Tall. *Algebraic number theory and Fermat's last theorem.* AK Peters/CRC Press, 2001.
34. Anat Talmy and Oren Dobzinski. Abuse freedom in access control schemes. In *20th International Conference on Advanced Information Networking and Applications-Volume 1 (AINA'06)*, volume 2, pages 77–86. IEEE, 2006.

35. Jacques Vélu. Isogénies entre courbes elliptiques. *Comptes Rendus de l'Académie des Sciences de Paris*, 273:238–241, July 1971.
36. William C Waterhouse. Abelian varieties over finite fields. In *Annales scientifiques de l'École normale supérieure*, volume 2, pages 521–560, 1969.
37. Jian Weng, Robert H Deng, Shengli Liu, and Kefei Chen. Chosen-ciphertext secure bidirectional proxy re-encryption schemes without pairings. *Information Sciences*, 180(24):5077–5089, 2010.
38. Keita Xagawa and Keisuke Tanaka. Proxy re-encryption based on learning with errors (mathematical foundation of algorithms and computer science). In *Proceedings of the 2010 Symposium on Cryptography and Information Security (SCIS 2010)*, 2010.

A Computation of Class group action on supersingular elliptic curves of $\mathbb{F}_p$

In practice, we cannot compute the class group action efficiently using Theorem 8. In Asiacrypt 2018, Castryck et al. [6] introduced Algorithm 7 described below to compute the group action efficiently for a particular case based on the following proposition proved by them.

Proposition 5 ([6]). *Let $p \geq 5$ be a prime such that $p \equiv 3 \pmod 8$ and let E be a supersingular elliptic curve over $\mathbb{F}_p$. Then $\mathsf{End}_{\mathbb{F}_p}(E) \cong \mathbb{Z}[\pi]$ if and only if there exists an element $A \in \mathbb{F}_p$ such that E is $\mathbb{F}_p$-isomorphic to the curve $E_A : y^2 = x^3 + Ax^2 + x$. Moreover, if such an A exists, then it is unique. The unique element A is called the Montgomery coefficient of the curve E_A and we denote it by $\mathsf{MC}(E_A) = A$.*

Let p be a large prime of the form $4\ell_1\ell_2 \cdots \ell_n - 1$ where ℓ_i's are small distinct odd primes. A base supersingular elliptic curve is set in Montgomery form $E_0 : y^2 = x^3 + x$ over $\mathbb{F}_p$. By Proposition 5, the endomorphism ring of E_0 over $\mathbb{F}_p$ is $\mathscr{O} = \mathsf{End}_{\mathbb{F}_p}(E_0) \cong \mathbb{Z}[\pi]$ where π is the Frobenius map defined in Definition 9. As trace $t = 0$ for supersingular elliptic curve and $p \equiv -1 \pmod{\ell_i}$, π satisfies $\pi^2 - 1 \equiv 0 \pmod{\ell_i}$ by Proposition 1 which in turn implies that the ideal $\ell_i\mathscr{O}$ splits as $\ell_i\mathscr{O} = \mathfrak{l}_i\bar{\mathfrak{l}}_i$ where $\mathfrak{l}_i = \langle \ell_i, \pi - 1\rangle$ and $\bar{\mathfrak{l}}_i = \langle \ell_i, \pi + 1\rangle$, i.e. $\mathfrak{l}_i^{-1} = \bar{\mathfrak{l}}_i$. Also, the kernel of the isogeny $\phi_{\mathfrak{l}_i}$ corresponding to the ideal $\mathfrak{l}_i$ is $\mathsf{ker}(\phi_{\mathfrak{l}_i}) = \mathsf{ker}([\ell_i]) \cap \mathsf{ker}(\pi - 1)$ and the kernel of the isogeny $\phi_{\bar{\mathfrak{l}}_i}$ corresponding to the ideal $\bar{\mathfrak{l}}_i$ is $\mathsf{ker}(\phi_{\bar{\mathfrak{l}}_i}) = \mathsf{ker}([\ell_i]) \cap \mathsf{ker}(\pi + 1)$. An integer s is chosen such that $(2s+1)^n \geq |\mathsf{cl}(\mathscr{O})|$. Then it holds that $|\mathsf{cl}(\mathscr{O})| \approx |\{[\mathfrak{l}_1^{e_1} \cdots \mathfrak{l}_n^{e_n} | e_1, \ldots, e_n \in \{-s, \ldots, s\}^n]\}|$ and it is possible to efficiently calculate the action of an ideal class $\prod_{i=1}^{n}[\mathfrak{l}_i^{e_i}]$ where $(e_1, \ldots, e_n) \in \{-s, \ldots, s\}^n$ by composing the action of the ideal classes $[\mathfrak{l}_i]$ or $[\mathfrak{l}_i^{-1}]$ depending on the signs of the exponents e_i using the polynomial time procedure $\mathsf{ComputeGroupAction}$ described in Algorithm 7.

Remark 4. It follows from the Proposition 4 that even if $p = 2^r\ell_1 \cdots \ell_n - 1$ where ℓ_i are small odd primes, one can compute the class group action of $\mathsf{cl}(\mathscr{O})$ on $\mathscr{E}_p(\mathscr{O})$ in the same way as in Algorithm 7.

Algorithm 7: $\mathsf{ComputeGroupAction}(p, \ell_1, \ldots, \ell_n, E_A : y^2 = x^3 + Ax^2 + x \in \mathscr{E}_p(\mathscr{O}), (e_1, \ldots, e_n) \in \{-s, \ldots, s\}^n) \rightarrow \mathsf{MC}([\mathfrak{a}]E_A)$ where $[\mathfrak{a}] = \prod_{i=1}^{n} [\mathfrak{l}_i^{e_i}], p = 4\ell_1\ell_2 \cdots \ell_n - 1$ and $\mathfrak{l}_i = \langle \ell_i, \pi - 1 \rangle$

1 Set $\mathbf{e} = (e_1, e_2, \ldots, e_n)$;
2 **while** $(\mathbf{e} \neq (0, 0, \ldots, 0))$ **do**
3   Sample $x \xleftarrow{\$} \mathbb{F}_p$;
4     $x(P) \leftarrow x$;
5   **if** ($x^3 + Ax^2 + x$ *is a square in* $\mathbb{F}_p$) **then**
6   Set $\alpha \leftarrow 1$;
7   **else**
8         $\alpha \leftarrow -1$;
9   **end if**
10   Set $\Lambda = \{j | \text{ sign}(e_j) = \text{sign}(\alpha) \text{ for } j \in [n]\}$;
11   **if** $(\Lambda = \emptyset)$ **then**
12   **goto** Step 2
13   **else**
14         $k \leftarrow \prod_{i \in \Lambda} \ell_i$;
15         $x(P) \leftarrow x(((p+1)/k)P)$;
16   **for** (***all*** $i \in \Lambda$) **do**
17             $x(Q) \leftarrow x((k/\ell_i)P)$;
18   **if** $(Q \neq O)$ **then**
19   Compute an ℓ_i-isogeny $\phi : E_A \rightarrow E_{A'}$ with $\mathsf{ker}\phi = \langle Q \rangle$ using Vélu's formula (Theorem 6);
20                 $A \leftarrow A'; x(P) \leftarrow x(\phi(P)); k \leftarrow k/\ell_i; e_i \leftarrow e_i - \alpha$;
21   **end if**
22   **end for**
23   **end if**
24 **end while**
25 **return** A